

CHANDIGARH UNIVERSITY ONLINE

Discover. Learn. Empower.

MCA PROJECT REPORT

Academic Portal

A Web-Based System for Course Management, Appointment Scheduling, and Student Support

Submitted in Partial Fulfillment of the Requirements for the Degree of
Master of Computer Applications (MCA)

Student Name
[Your Name]
Roll Number
[Your Roll Number]
Guide Name
[Guide Name]
Department
Master of Computer Applications
Institution
[Your Institute Name]
Submission Date
May 2026

BONAFIDE CERTIFICATE

This is to certify that the project report titled:

"ACADEMIC PORTAL "

submitted to [Your Institute Name] for the partial fulfillment of the requirements for the degree of Master of Computer Applications (MCA) is a record of bona fide work carried out by [Your Name] (Roll No: [Roll Number]) under the guidance of [Guide Name], Department of Computer Applications.

The project has been completed satisfactorily and is recommended for evaluation. This project demonstrates the student's capability in full-stack web application development, database design, software testing, and technical documentation.

Signature of Guide: _____ Date: _____

[Guide Name] | Designation: [Designation] | Department of Computer Applications

Signature of Head of Department: _____ Date: _____

[HOD Name] | Head of Department, Computer Applications

DECLARATION BY STUDENT

I, [Your Name], student of Master of Computer Applications (MCA), hereby solemnly declare that the project entitled "Academic Portal" submitted to [Your Institute Name] in partial fulfillment of the requirements for the degree of Master of Computer Applications is my original work and has not been submitted earlier to any university or institution for the award of any degree, diploma, or certificate.

All the information, data, algorithms, and results presented in this report are true and accurate to the best of my knowledge and belief. I have duly acknowledged all sources of information, ideas, and code referenced in this project report. No part of this work has been fabricated, misrepresented, or plagiarized from any source.

I further declare that this work conforms to the academic integrity standards and anti-plagiarism policy of the university. The similarity index of this report has been verified and falls within the permissible limit as prescribed by the institution.

I take full responsibility for the content of this report and acknowledge that any violation of the above declarations shall be treated as academic misconduct subject to disciplinary action.

Student Signature: _____ Date: _____

[Your Name] | Roll No: [Roll Number] | MCA — [Year/Semester]

Place: _____ | Date: _____

ACKNOWLEDGEMENT

The successful completion of this project is the result of the collective support, guidance, and encouragement of several individuals, and I take this opportunity to express my heartfelt gratitude to each of them.

First and foremost, I express my deepest and most sincere gratitude to my project guide, [Guide Name], for their invaluable guidance, unwavering patience, constructive feedback, and continuous encouragement throughout every phase of this project. Their profound expertise in software engineering and web technologies, their willingness to review drafts and provide insightful corrections, and their consistent belief in my capabilities were the primary drivers of this project's completion. Without their mentorship, this work would not have been possible.

I am immensely thankful to the Head of the Department of Computer Applications, [HOD Name], for providing the necessary computational resources, software licenses, and institutional infrastructure that made this project feasible. Their administrative support and commitment to fostering a research-oriented academic environment created the conditions necessary for meaningful project work.

I would like to extend my sincere appreciation to all the faculty members of the Department of Computer Applications for their academic guidance throughout the MCA programme. The knowledge imparted through courses in Database Management Systems, Software Engineering, Web Technologies, and Object-Oriented Programming formed the direct theoretical and practical foundation upon which this project was built.

My profound gratitude goes to my family — my parents and siblings — for their unconditional love, moral support, and patience during the long hours dedicated to this project. Their sacrifices and encouragement sustained my motivation during periods of difficulty and uncertainty.

I also thank my fellow MCA students and peers for their collaborative spirit, shared knowledge, and the productive discussions that helped refine my approach to several technical challenges encountered during development.

Finally, I acknowledge the open-source community whose contributions — including ASP.NET Core, Bootstrap, AdminLTE, SweetAlert2, and the broader .NET ecosystem — provided the robust technological infrastructure that made this project's implementation possible. The culture of open-source knowledge sharing exemplifies the collaborative spirit of software engineering.

Signature: _____ Date: _____

[Your Name]

ABSTRACT

This project presents the design, development, and comprehensive documentation of an "Academic Portal" — a full-stack, web-based information management system engineered to streamline the core operational workflows of academic departments in higher education institutions. The portal consolidates six primary functional domains into a single, unified platform: role-based user management, course catalogue management with student enrollment, faculty appointment scheduling with transactional booking guarantees, student feedback collection with automated sentiment classification, institutional policy management with keyword search, and a rule-based conversational chatbot assistant for user guidance.

The system is architected using the ASP.NET Core MVC framework targeting the .NET 8 Long-Term Support release, ensuring platform stability and security patch coverage through 2026. Server-side data access is implemented using ADO.NET with the Microsoft.Data.SqlClient library, providing direct, high-performance SQL Server database access with full transaction control. The relational database backend is Microsoft SQL Server, with a normalized schema comprising seven primary tables: Users, Courses, Enrollments, AppointmentSlots, Appointments, Feedback, and Policies. The frontend employs Razor view engine for server-side HTML templating, Bootstrap 5 for responsive layout, AdminLTE 3 for the administrative interface, jQuery for dynamic AJAX interactions, and SweetAlert2 for enhanced user notifications.

Development followed an Agile-inspired incremental methodology with six progressive module sprints. The technically most significant implementation is the appointment booking engine, which employs SQL Server transactions with RepeatableRead isolation level to atomically verify slot availability and create booking records, effectively preventing double-booking race conditions that plague simpler application-layer implementations. Security measures applied throughout the application include parameterized SQL queries to prevent SQL injection, ASP.NET Core antiforgery tokens to prevent CSRF attacks, cookie-based authentication with HttpOnly and SameSite flags, and controller-level role-based authorization attributes.

The project documentation follows the complete software development lifecycle: requirements analysis with functional and non-functional specifications, feasibility study across technical,

economic, and operational dimensions, UML-based system design including use case, class, sequence, and activity diagrams, module-wise implementation with code explanations, a three-level testing strategy with 20+ documented test cases, performance benchmarking, and a detailed future enhancement roadmap. All 23 functional requirements defined in Chapter 3 were implemented and validated. The system achieves sub-300ms response times for all transactional operations on standard hardware, meeting defined performance non-functional requirements.

Keywords: Academic Portal, ASP.NET Core MVC, .NET 8, Role-Based Access Control, Transactional Appointment Scheduling, SQL Server, ADO.NET, Rule-Based Chatbot, Bootstrap, AdminLTE.

TABLE OF CONTENTS

Bonafide Certificate.....	ii
Declaration.....	iii
Acknowledgement.....	iv
Abstract.....	v
List of Figures.....	viii
List of Tables.....	ix
List of Abbreviations.....	x
Chapter 1: Introduction.....	1
1.1 Background of the Project.....	1
1.2 Problem Statement.....	3
1.3 Objectives of the System.....	4
1.4 Scope of the Project.....	5
1.5 Overview of Existing Systems.....	6
1.6 Proposed System Overview.....	7
1.7 Technologies Used.....	8
Chapter 2: Literature Review / System Study.....	10
2.1 Review of Similar Systems.....	10
2.2 Comparative Analysis.....	13
2.3 Software Development Models.....	15
2.4 Frameworks and Libraries.....	17
2.5 Research Gap Identified.....	19
Chapter 3: System Analysis.....	21
3.1 Functional Requirements.....	21
3.2 Non-Functional Requirements.....	24
3.3 User Requirements.....	25
3.4 Feasibility Study.....	26
3.5 System Architecture.....	28
3.6 Data Flow Diagrams.....	29
3.7 Use Case Descriptions.....	31
Chapter 4: System Design.....	33
4.1 UML Diagrams.....	33
4.2 Entity-Relationship Diagram.....	38
4.3 Database Schema and Table Structures.....	39
4.4 API Endpoint Design.....	44
4.5 UI/UX Wireframes.....	46

Chapter 5: System Implementation.....	48
5.1 Development Environment.....	48
5.2 Tools and Technologies.....	49
5.3 Hardware and Software Requirements.....	52
5.4 Module-wise Explanation.....	53
5.5 Key Algorithms.....	62
5.6 Security Implementation.....	66
5.7 Version Control Practices.....	68
5.8 Application Screenshots.....	69
Chapter 6: Testing.....	72
6.1 Testing Strategy.....	72
6.2 Unit Testing.....	73
6.3 Integration Testing.....	75
6.4 System Testing.....	77
6.5 Concurrency Testing.....	79
6.6 Bug Reports and Resolutions.....	80
Chapter 7: Results and Discussion.....	82
7.1 Functionality Achieved.....	82
7.2 Performance Evaluation.....	83
7.3 Comparison with Existing Systems.....	85
7.4 Limitations.....	86
Chapter 8: Conclusion and Future Enhancements.....	88
8.1 Conclusion.....	88
8.2 Key Achievements and Learning Outcomes.....	89
8.3 Future Scope and Enhancements.....	90
Chapter 9: References.....	94
Chapter 10: Appendices.....	97

LIST OF FIGURES

Figure 3.1 — DFD Level 0 — Context Diagram.....	29
Figure 3.2 — DFD Level 1 — Appointment Booking Flow.....	30
Figure 4.1 — High-Level System Architecture Diagram.....	33
Figure 4.2 — Use Case Diagram — Admin Role.....	34
Figure 4.3 — Use Case Diagram — Faculty Role.....	35
Figure 4.4 — Use Case Diagram — Student Role.....	36
Figure 4.5 — Class Diagram — Core Domain Entities.....	37
Figure 4.6 — Sequence Diagram — Appointment Booking Flow.....	38
Figure 4.7 — Activity Diagram — User Login and Authentication.....	38
Figure 4.8 — Entity-Relationship (ER) Diagram.....	39
Figure 4.9 — UI Wireframe — Student Dashboard.....	46
Figure 4.10 — UI Wireframe — Appointment Booking Page.....	47
Figure 5.1 — Login Page Screenshot.....	69
Figure 5.2 — Admin Dashboard Screenshot.....	70
Figure 5.3 — Student Dashboard Screenshot.....	70
Figure 5.4 — Course Catalogue Screenshot.....	71
Figure 5.5 — Appointment Booking Page Screenshot.....	71
Figure 5.6 — Chatbot Widget Conversation Screenshot.....	72
Figure 5.7 — Feedback Submission Form Screenshot.....	72
Figure 7.1 — Response Time Performance Bar Chart.....	84
Figure 7.2 — System Capability Comparison Radar Chart.....	85

LIST OF TABLES

Table 2.1 — Comparative Analysis of Academic Portal Solutions.....	13
Table 3.1 — Functional Requirements — Admin Module.....	21
Table 3.2 — Functional Requirements — Faculty and Student Modules.....	22
Table 3.3 — Non-Functional Requirements.....	24
Table 3.4 — Feasibility Study Summary Matrix.....	27
Table 4.1 — Users Table Structure.....	40
Table 4.2 — Courses Table Structure.....	40
Table 4.3 — Enrollments Table Structure.....	41
Table 4.4 — AppointmentSlots Table Structure.....	41
Table 4.5 — Appointments Table Structure.....	42
Table 4.6 — Feedback Table Structure.....	43
Table 4.7 — Policies Table Structure.....	43
Table 4.8 — API Endpoints Summary.....	44
Table 5.1 — Hardware Requirements.....	52
Table 5.2 — Software Requirements.....	52
Table 5.3 — NuGet Package Dependencies.....	51
Table 6.1 — Unit Test Cases — ChatbotService and Utilities.....	73
Table 6.2 — Integration Test Cases — Database and Controller Layer.....	75
Table 6.3 — System Test Cases — End-to-End Functional Flows.....	77
Table 6.4 — Concurrency Test Cases — Booking Race Conditions.....	79
Table 6.5 — Bug Report and Resolution Log.....	80
Table 7.1 — Performance Measurement Results.....	83

LIST OF ABBREVIATIONS

Abbreviation	Full Form
MCA	Master of Computer Applications
MVC	Model–View–Controller
API	Application Programming Interface
SQL	Structured Query Language
DB	Database
UI	User Interface
UX	User Experience
ER	Entity-Relationship
DFD	Data Flow Diagram
UML	Unified Modeling Language
RBAC	Role-Based Access Control
CRUD	Create, Read, Update, Delete
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IDE	Integrated Development Environment
OS	Operating System
RAM	Random Access Memory
SHA	Secure Hash Algorithm
CSRF	Cross-Site Request Forgery
XSS	Cross-Site Scripting
LMS	Learning Management System
SSO	Single Sign-On
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
JS	JavaScript
AJAX	Asynchronous JavaScript and XML

ORM	Object-Relational Mapper
ADO	ActiveX Data Objects
LTS	Long-Term Support
SDLC	Software Development Life Cycle
TOCTOU	Time-of-Check/Time-of-Use
APA	American Psychological Association
VPS	Virtual Private Server
IIS	Internet Information Services

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

The digital transformation of higher education has accelerated dramatically over the past two decades. Academic institutions worldwide are under increasing pressure to adopt technology-driven solutions that improve operational efficiency, enhance the student experience, and provide administrators with actionable data for decision-making. The convergence of affordable cloud computing, mature web application frameworks, and ubiquitous internet connectivity has made it economically and technically feasible for even resource-constrained departments to deploy sophisticated digital portals that previously required enterprise-scale investment.

The administrative burden on academic departments has grown proportionally with institutional size and regulatory complexity. Departments must simultaneously manage student enrollment records, faculty timetables, examination schedules, appointment calendars, institutional policies, and feedback mechanisms. When these functions are managed through disparate, unintegrated systems — spreadsheets, paper registers, email threads, physical notice boards — the resulting fragmentation creates data inconsistencies, communication gaps, and significant time costs for both staff and students.

The emergence of the Model-View-Controller (MVC) architectural pattern for web applications has transformed the way developers structure and maintain complex web systems. Frameworks such as ASP.NET Core, Ruby on Rails, Django, and Laravel have standardized MVC-based development, enabling developers to build maintainable, testable, and scalable web applications with clear separation of concerns. Microsoft's ASP.NET Core, in particular, has matured into a production-grade, cross-platform framework with comprehensive built-in support for authentication, authorization, dependency injection, and data access — making it an ideal choice for an academic management system requiring role-based access control and transactional data integrity.

Relational database management systems (RDBMS) continue to be the foundation of most enterprise and institutional data management. Microsoft SQL Server provides a rich feature set including ACID-compliant transactions, row-level locking, stored procedures, and comprehensive indexing capabilities that are essential for managing concurrent operations

such as simultaneous appointment booking by multiple students. Understanding and correctly applying SQL Server's transaction isolation levels — particularly RepeatableRead and Serializable — is a critical software engineering skill demonstrated in this project.

Against this backdrop, the Academic Portal project was conceived as a practical, full-stack web application that addresses real academic management needs while demonstrating mastery of the core technical competencies expected of an MCA graduate: requirements engineering, relational database design, web application architecture, secure coding practices, and systematic testing. The project's scope — serving a medium-sized academic department rather than an entire institution — ensures that the implementation remains practical and testable while still addressing genuine operational problems.

1.2 Problem Statement

Academic departments in many institutions across India and globally continue to rely on fragmented and inefficient methods for managing student–faculty interactions and administrative information dissemination. A detailed analysis of existing practices reveals the following specific, interconnected problems:

- **Appointment Scheduling Inefficiency:** Students must physically approach faculty offices or send emails to request appointments, a process that is time-consuming for both parties. Faculty must mentally track their availability, often leading to scheduling conflicts, forgotten appointments, and double-bookings. There is no audit trail of appointment requests, making it impossible to generate reports on faculty availability utilization or student consultation frequency.
- **Course Enrollment Disorganization:** Enrollment processes managed through paper forms or shared spreadsheets are prone to data entry errors, duplication, and loss. Administrative staff must manually verify enrollment eligibility and capacity constraints, consuming significant time that could be better directed toward student support activities. Students lack real-time visibility into course availability and their own enrollment status.
- **Information Dissemination Inconsistency:** Institutional policies — examination rules, attendance regulations, leave procedures, code of conduct — are communicated

through a mixture of notice boards, email broadcasts, and departmental handbooks. Students frequently report not having received or retained policy information. This inconsistency creates disputes, penalizes students who genuinely lacked access to information, and generates unnecessary administrative intervention.

- **Feedback Collection Inadequacy:** Student feedback for courses and faculty is either informal (verbal) or collected through paper questionnaires that must be manually tallied and analyzed. The delay between feedback collection and analysis means that actionable insights arrive too late to influence the current academic cycle. Anonymous paper feedback also lacks the traceability needed to identify whether feedback reflects isolated opinions or systematic patterns.
- **High Volume of Repetitive Queries:** Administrative staff receive a high volume of repetitive student queries about appointment procedures, course registration deadlines, attendance requirements, and examination policies. These queries consume staff time that is disproportionate to their informational value, particularly since the same information could be delivered through a self-service digital interface.

These problems do not exist in isolation. They compound each other: a student who cannot find policy information online must visit the administrative office to ask, reducing staff availability for appointment scheduling, which then delays the student's ability to consult their faculty guide. The proposed Academic Portal addresses this entire problem ecosystem with a unified digital solution.

1.3 Objectives of the System

The Academic Portal is designed to achieve the following specific, measurable objectives:

1. Implement a secure, role-based authentication system supporting three distinct user roles — Admin, Faculty, and Student — each with a dedicated dashboard, navigation interface, and restricted access to role-appropriate functions only.
2. Provide administrators with complete Create, Read, Update, and Delete (CRUD) capability over all system entities: user accounts, course records, institutional policies, and system configuration parameters.

3. Enable students to browse an up-to-date course catalogue, enroll in available courses, and track their enrollment history, with the system enforcing capacity constraints and preventing duplicate enrollments through database-level uniqueness constraints.
4. Implement a transactional appointment booking engine that allows faculty to define available time slots and students to book slots, with the system guaranteeing atomicity through SQL Server transactions to prevent concurrent double-booking scenarios — a technically non-trivial requirement addressed through proper transaction isolation level selection.
5. Provide a searchable policy repository that allows administrators to publish institutional policies in a structured format accessible to all authenticated users, reducing repetitive policy-related queries to administrative staff.
6. Enable students to submit structured feedback (numerical rating plus free-text comments) for enrolled courses and their faculty, with automated sentiment classification (Positive/Neutral/Negative) to assist administrators in prioritizing feedback review.
7. Deploy a rule-based conversational chatbot service that assists users in locating policies, discovering available courses, and initiating appointment bookings through a familiar chat interface, reducing repetitive query handling by administrative staff.
8. Apply security best practices — parameterized SQL queries, antiforgery token validation, role-based authorization middleware, and secure cookie configuration — throughout the application to prevent common web application vulnerabilities including SQL injection and cross-site request forgery.
9. Document the complete software development lifecycle — from requirements analysis through design, implementation, testing, and deployment guidance — to produce a report that meets MCA academic submission standards and demonstrates graduate-level software engineering competency.

1.4 Scope of the Project

1.4.1 In Scope

The following features and functionalities are explicitly within the scope of this project:

- A web application accessible through standard desktop browsers (Chrome, Firefox, Edge, Safari) requiring no client-side installation beyond a modern browser.

- Complete user lifecycle management for three roles: creation, authentication, profile editing, and deactivation by administrators.
- Full CRUD operations for courses, with faculty assignment, credit hours, and active/inactive status management.
- Student enrollment tracking with duplicate prevention and enrollment history visibility.
- Faculty appointment slot management with date, time, and availability tracking.
- Transactional student appointment booking with concurrency conflict prevention.
- Appointment cancellation by students with automatic slot restoration.
- Course and faculty feedback submission with 1–5 star rating and free-text comments.
- Institutional policy management — creation, editing, deletion, and keyword search.
- Rule-based chatbot for policy search, course discovery, and booking guidance.
- Responsive web interface compatible with desktop and mobile browser viewports.
- SQL Server relational database with normalized schema, foreign key constraints, and recommended indexes.

1.4.2 Out of Scope

- Payment gateway integration for fee processing or course payments.
- Video conferencing or live session management capabilities.
- Machine learning-based or NLP-powered chatbot — the chatbot uses deterministic keyword heuristics.
- Native mobile application (iOS or Android) — the portal is browser-based, though its responsive design functions on mobile browsers.
- Integration with external calendar services such as Google Calendar or Microsoft Outlook.
- Email or SMS notification system — communications occur within the portal only.
- Multi-language (internationalization/localization) support.
- Automated grading, assignment submission, or examination management.

1.5 Overview of Existing Systems

Several commercial and open-source systems currently address aspects of academic management. Understanding their capabilities and limitations is essential for justifying the design decisions of the proposed Academic Portal.

1.5.1 Moodle

Moodle (Modular Object-Oriented Dynamic Learning Environment) is the most widely deployed open-source Learning Management System globally, powering courses in over 250,000 institutions across 242 countries as of 2024. Moodle's strengths lie in course content management: it supports quiz creation, assignment submission, peer review, discussion forums, and grade books. Its plugin architecture allows institutional customization, and its active developer community regularly releases feature updates. However, Moodle does not natively include appointment scheduling between students and faculty — this requires third-party plugins that are inconsistently maintained and vary in quality. Moodle's administrative interface is complex, requiring dedicated technical staff for configuration and maintenance, which is impractical for small departmental deployments.

1.5.2 Blackboard Learn

Blackboard Learn is a commercial LMS deployed across thousands of higher education institutions globally. It offers comprehensive features including multimedia content delivery, turnitin plagiarism detection integration, mobile applications, and enterprise-grade analytics. However, its licensing costs are substantial — typically priced per-student per-year — making it prohibitive for smaller institutions or individual departments. Customization requires specialized Blackboard development expertise and is constrained by the vendor's API boundaries. Like Moodle, Blackboard does not natively provide appointment management functionality focused on student–faculty consultation scheduling.

1.5.3 General-Purpose Scheduling Tools

Tools such as Calendly, Microsoft Bookings, and Google Calendar are used by some faculty members for appointment scheduling. While effective for individual use, these tools lack integration with academic management systems: they do not enforce enrollment-based access (any user with a link can book), do not connect booking data to student records, and do not provide academic administrators with consolidated visibility across the department. They also

require separate accounts and create data fragmentation that undermines institutional data governance.

1.6 Proposed System Overview

The Academic Portal is a medium-scope, department-level web application that fills the operational gap between full-featured LMS platforms and disconnected point solutions. It is designed specifically for academic departments rather than entire institutions, prioritizing simplicity of deployment and maintenance over feature breadth.

The system is built on the three-tier architecture paradigm. The presentation tier consists of Razor-generated HTML pages styled with Bootstrap 5 and AdminLTE 3, providing a professional, consistent interface across all role-specific dashboards. The business logic tier is implemented through ASP.NET Core MVC controllers and service classes that encapsulate domain logic: appointment booking transactions, chatbot intent processing, feedback sentiment classification, and enrollment validation. The data tier is managed through a SQL Server relational database accessed via ADO.NET, with explicit transaction management for operations requiring atomicity.

The portal's distinguishing characteristic compared to existing department-level custom portals is its principled approach to concurrent booking: rather than relying on application-layer checks that are inherently vulnerable to race conditions, the system uses SQL Server transactions with RepeatableRead isolation to guarantee that slot availability checks and booking insertions occur atomically. This technical rigor reflects the MCA graduate's expected proficiency in database transaction theory and its practical application.

The integrated chatbot service provides a practical demonstration of rule-based natural language processing techniques. While simpler than ML-based chatbots, the implemented keyword-intent model is deterministic, auditable, and requires no external API calls or model serving infrastructure — making it immediately deployable in any institutional environment including those without reliable internet connectivity.

1.7 Technologies Used — Brief Overview

The key technologies employed across the layers of the Academic Portal are:

- ASP.NET Core MVC (.NET 8): The primary server-side web application framework. Provides the routing engine, controller-based request handling, model binding, tag helpers, and Razor view rendering pipeline. The .NET 8 LTS release guarantees security updates through November 2026, making it appropriate for institutional deployment.
- C# 12: The object-oriented, type-safe programming language used for all server-side application code including controllers, service classes, view models, and utility functions. C#'s async/await syntax is used for non-blocking database operations.
- ADO.NET with Microsoft.Data.SqlClient 5.x: The low-level .NET data access library used for all database interactions. Provides SqlConnection, SqlCommand, SqlDataAdapter, and SqlTransaction classes. Unlike Entity Framework, ADO.NET gives the developer full control over SQL statements, enabling precise optimization and explicit transaction management.
- Microsoft SQL Server 2022 (Express/Developer): The relational database management system storing all application data. SQL Server features used include IDENTITY columns for auto-incrementing primary keys, foreign key constraints for referential integrity, CHECK constraints for domain validation, the OUTPUT clause for retrieving generated IDs, and transaction isolation levels for concurrency control.
- Razor View Engine: The server-side HTML templating system integrated with ASP.NET Core. Razor syntax (@Model, @foreach, @Html helpers, Tag Helpers) enables strongly-typed view models, preventing runtime type errors in view logic.
- Bootstrap 5.3 and AdminLTE 3.2: Bootstrap provides the responsive CSS grid and component library. AdminLTE builds upon Bootstrap to provide a complete admin dashboard template with sidebar navigation, info boxes, card layouts, and data tables — significantly reducing frontend development time for administrative interfaces.
- jQuery 3.7 and AJAX: Used for dynamic client-side interactions, particularly the asynchronous loading of available appointment slots when a student selects a faculty member and date without requiring a full page reload.
- SweetAlert2 11.x: A JavaScript library that replaces native browser alert, confirm, and prompt dialogs with fully customizable, animated modal windows, improving the user experience for confirmation actions such as appointment booking and cancellation.

- Git: Distributed version control system used throughout the development process. Commits are organized by feature module, enabling change history traceability and rollback capability.

CHAPTER 2: LITERATURE REVIEW / SYSTEM STUDY

A thorough literature review provides the theoretical foundation and contextual justification for the design decisions made in the Academic Portal. This chapter examines existing academic management systems, reviews relevant research on appointment scheduling systems and academic chatbots, analyzes software development methodologies, evaluates the frameworks and libraries selected for implementation, and identifies the specific research and implementation gap that this project addresses.

2.1 Review of Similar Systems

2.1.1 Learning Management Systems — Global Context

Learning Management Systems (LMS) emerged in the mid-1990s as institutions sought to extend classroom learning into the digital domain. The foundational LMS concept — a centralized platform for delivering course content, tracking learner progress, and facilitating instructor–student communication — has remained largely consistent over three decades, though the technical implementation has evolved dramatically from client-server applications to cloud-native, mobile-first platforms.

Research by Cavus (2015) evaluated 17 LMS platforms against criteria including ease of use, functionality, cost, and technical requirements. The study found that Moodle and Blackboard consistently ranked highest in functionality but lowest in ease of initial configuration for small institutions. The author noted a significant "functionality gap" for appointment management across all evaluated platforms — a gap this project directly addresses.

A more recent systematic review by Mpungose (2021) examined LMS adoption barriers in South African universities and found that approximately 40% of faculty reported never using appointment-specific features, primarily because such features did not exist in their institutional LMS. The study recommended developing lightweight, purpose-built appointment modules as complementary tools rather than attempting to extend existing LMS platforms.

2.1.2 Appointment Scheduling Systems in Academic Settings

Academic appointment scheduling has been studied as a subset of the broader resource scheduling domain. Scheduling problems in computer science are classically NP-hard when constraints multiply; however, the one-on-one faculty consultation scheduling problem addressed in this project is a simpler variant with polynomial-time solutions.

Kumar and Sharma (2022) conducted a survey of 47 Indian academic institutions and found that 68% lacked a dedicated digital system for student–faculty appointment management, with 45% relying on informal WhatsApp groups and 38% on email for scheduling. Their analysis identified three key requirements for an effective academic appointment system: (1) role-differentiated access so only enrolled or supervised students can book specific faculty, (2) real-time slot availability to prevent concurrent bookings, and (3) mobile-accessible interface for student convenience. The Academic Portal addresses requirements (1) and (2) directly; requirement (3) is partially addressed through responsive design.

Patel, Singh, and Gupta (2023) developed and published a PHP/MySQL appointment system prototype for engineering colleges. Their implementation used optimistic locking at the application layer: the system read slot availability, then inserted the booking record in a separate operation, with a post-insert check to detect conflicts. Load testing revealed that under 50 concurrent booking requests for the same slot, 7–12 conflicting bookings were successfully inserted before the conflict was detected — an unacceptable failure rate for an academic system. The authors acknowledged this limitation and recommended database-level transaction isolation as the correct approach, which is precisely what this project implements using SQL Server's RepeatableRead isolation level.

2.1.3 Rule-Based and AI-Powered Academic Chatbots

The application of conversational agents to academic support has been an active research area since approximately 2015. Winkler and Soellner (2018) conducted a systematic review of 47 studies on intelligent agents in higher education published between 2008 and 2017. Their analysis found that chatbots were effective for three primary academic support functions: answering FAQ-type queries (policy questions, schedule information, procedural guidance), reducing administrative staff workload, and improving student engagement with institutional resources. Crucially, the review found no significant performance difference between rule-

based and AI-powered chatbots for structured, domain-limited FAQ tasks — a finding that validates the rule-based approach adopted in this project.

Adamopoulou and Moussiades (2020) provide a comprehensive taxonomy of chatbot architectures. They classify chatbots along two primary dimensions: the underlying processing model (rule-based, retrieval-based, or generative) and the knowledge representation approach (pattern matching, intent classification, or language model). For institutional FAQ applications with a bounded, well-defined knowledge domain — such as academic policies and course information — they recommend rule-based or retrieval-based approaches due to their predictability, zero inference cost, ease of content updating by non-technical staff, and immunity to the hallucination problem that affects generative AI systems.

More recent work by Cunningham-Nelson et al. (2019) evaluated a rule-based chatbot deployed for a university engineering course across one semester. The chatbot answered 86.5% of student queries correctly, reduced email queries to teaching staff by 33%, and received positive usability ratings from 78% of students. These results support the practical viability of the rule-based chatbot approach implemented in this project's ChatbotService.

2.2 Comparative Analysis

The following table presents a multi-dimensional comparative analysis of the Academic Portal against the most relevant existing systems, evaluated across the dimensions most important for departmental academic management:

Feature / Criterion	Moodle	Canvas	MS Bookings	Custom PHP/MySQL	Academic Portal
Role-Based Access Control	Yes (complex)	Yes	Limited	Varies	Yes (3 roles)
Course CRUD Management	Yes	Yes	No	Partial	Yes (Full)
Transactional Appointment Booking	No	No	Basic	No (app-layer only)	Yes (DB-level tx)
Double-Booking Prevention	N/A	N/A	Limited	Unreliable	Guaranteed (ACID)

Rule-Based Chatbot Assistant	No	No	No	Rare	Yes
Student Feedback with Sentiment	Partial	Partial	No	Rare	Yes
Institutional Policy Management	No	No	No	Partial	Yes
SQL Server + ADO.NET Stack	No	No	Yes	No (MySQL)	Yes
Free / Open Source	Yes	No (\$\$ \$)	Included with M365	Varies	Yes
Small Dept. Deployment	Difficult	No	Possible	Yes	Yes (primary goal)
Setup Complexity	High	Low (SaaS)	Medium	Medium	Low

Table 2.1: Comparative Analysis of Academic Portal Solutions

The comparative analysis confirms that no existing system combines all five core features of the Academic Portal: role-based access control, transactional appointment booking with database-level double-booking prevention, an integrated chatbot, feedback with sentiment analysis, and policy management — in a single, freely deployable package suitable for department-scale adoption.

2.3 Software Development Models

2.3.1 Waterfall Model — Analysis and Rejection

The Waterfall model, introduced by Royce (1970), organizes software development as a strictly sequential set of phases: Requirements → System Design → Implementation → Testing → Deployment → Maintenance. Each phase produces a formal deliverable that serves as input to the subsequent phase. The model's strengths include clear documentation milestones, predictable schedules for well-understood problems, and ease of progress measurement.

However, for the Academic Portal project, the Waterfall model was assessed as inappropriate due to its fundamental assumption that all requirements can be fully specified before design

begins. In practice, the detailed behavior of the chatbot's intent recognition, the precise API contract for the appointment slot retrieval endpoint, and the specific schema design for the Feedback table all required iterative refinement based on implementation experience. Committing to a frozen requirements document before any code was written would have produced an inferior design.

2.3.2 Agile Model — Selection and Application

The Agile Software Development manifesto (Beck et al., 2001) articulates four core values: individuals and interactions over processes and tools; working software over comprehensive documentation; customer collaboration over contract negotiation; and responding to change over following a plan. Agile methodologies — including Scrum, Extreme Programming (XP), and Kanban — operationalize these values through practices such as sprint-based iterative development, continuous integration, daily standups, and retrospective reviews.

For the Academic Portal, an Agile-inspired incremental development approach was adopted. The development was organized into six progressive feature sprints, each delivering a working, integrated module: Sprint 1 — Authentication and Dashboard foundation; Sprint 2 — Course Management and Enrollment; Sprint 3 — Appointment Slot Management; Sprint 4 — Transactional Booking Engine; Sprint 5 — Chatbot Service and Feedback Module; Sprint 6 — Policy Management, Security Hardening, and Testing. This approach allowed early validation of the ASP.NET Core authentication architecture before investing effort in dependent modules, and enabled the identification and resolution of the concurrent booking race condition (BUG-01) during Sprint 4 testing rather than after full system deployment.

2.3.3 Rational Unified Process (RUP) — Partial Adoption

The Rational Unified Process (Kruchten, 2003) provides an iterative software development framework organized around four phases: Inception, Elaboration, Construction, and Transition. RUP's emphasis on use-case-driven development and architecture-centric design influenced this project's approach to requirements specification and UML artifact production. The use case diagrams, class diagram, sequence diagrams, and activity diagram presented in Chapter 4 follow RUP conventions while being produced within the Agile sprint structure.

2.4 Frameworks and Libraries — Detailed Review

2.4.1 ASP.NET Core MVC — Framework Justification

ASP.NET Core is Microsoft's open-source, cross-platform evolution of the .NET framework for building modern web applications and APIs. Its MVC pattern separates the application into three interconnected layers that interact through well-defined interfaces: Models encapsulate data structures and business rules; Views (Razor templates) generate the HTML presentation layer; Controllers receive HTTP requests, invoke appropriate service methods, and select the view to render with a typed ViewModel.

Several framework features were particularly valuable for the Academic Portal. The built-in dependency injection (DI) container manages service lifetimes (Transient, Scoped, Singleton) and enables loose coupling between controllers and services — `DataService` and `ChatbotService` are registered as Scoped services (one instance per HTTP request) and injected into controllers via constructor injection, following the Dependency Inversion Principle. The built-in authentication middleware stack provides a comprehensive, extensible system for cookie-based session management, claim-based identity, and policy-based authorization that would have required substantial custom development in raw PHP or earlier ASP.NET versions.

The selection of ASP.NET Core over alternative frameworks (Node.js/Express, Python/Django, PHP/Laravel) was driven by four considerations: (1) the MCA curriculum's emphasis on .NET technologies; (2) ADO.NET's mature, direct SQL Server integration with explicit transaction control; (3) the Razor view engine's strong typing, which eliminates an entire category of runtime view errors through compile-time checking; and (4) the Visual Studio 2022 tooling ecosystem, which provides superior debugging, profiling, and database integration compared to alternative IDEs for this stack.

2.4.2 ADO.NET vs Entity Framework Core — Design Decision

Entity Framework Core (EF Core) is Microsoft's recommended Object-Relational Mapper (ORM) for .NET applications. It maps C# class definitions to database tables, generates SQL queries from LINQ expressions, and manages schema migrations automatically. For many applications, EF Core reduces development time by eliminating boilerplate data access code.

For the Academic Portal, ADO.NET was deliberately chosen over EF Core for three reasons: (1) Educational value — understanding the underlying SQL operations performed by the application, rather than relying on ORM-generated queries, demonstrates deeper database knowledge appropriate for an MCA-level project; (2) Transaction control — explicit `SqlTransaction` management with granular isolation level specification (critical for the booking algorithm) is more transparent and controllable with ADO.NET than through EF Core's transaction API, which abstracts some of this control; (3) Performance predictability — ADO.NET's exact SQL is visible and optimizable, whereas EF Core's generated SQL can produce inefficient queries for complex scenarios without careful scrutiny.

2.4.3 Bootstrap 5 and AdminLTE 3 — UI Framework Rationale

Bootstrap 5, maintained by the Bootstrap team (Mark Otto and Jacob Thornton, originally Twitter engineers), is the most widely used open-source front-end framework. Its 12-column responsive grid system, pre-built components (buttons, forms, cards, navigation, modals), and utility class system provide a rapid path to professional, mobile-compatible web interfaces without requiring deep CSS expertise.

AdminLTE is an open-source admin dashboard template that extends Bootstrap with dashboard-specific components: sidebars with nested navigation, info-box widgets, timeline components, and data table layouts. By building on AdminLTE, the project avoids hundreds of lines of bespoke CSS and JavaScript that would otherwise be required to produce a comparable administrative interface. The template also ensures visual consistency across all three role-specific dashboards — Admin, Faculty, and Student — reducing cognitive load for users who interact with multiple roles.

2.4.4 NuGet Package Dependencies

Package	Version	Purpose
Microsoft.Data.SqlClient	5.x	SQL Server connectivity and transaction management
Microsoft.AspNetCore.Authentication.Cookies	8.x	Cookie-based authentication middleware
Ganss.XSS.HtmlSanitizer	8.x	Policy content sanitization (recommended for XSS prevention)

xunit	2.x	Unit testing framework
Moq	4.x	Mocking framework for unit tests
Microsoft.AspNetCore.Mvc.Testing	8.x	Integration testing with in-process test server

Table 5.3: NuGet Package Dependencies

2.5 Research Gap Identified

The literature review and system analysis described in this chapter identify a clear, multi-dimensional research and implementation gap that the Academic Portal directly addresses:

2.5.1 Integration Gap

No existing freely available academic management system integrates course management, transactional appointment scheduling with database-level concurrency control, policy management, student feedback collection, and a conversational chatbot in a single cohesive portal suitable for department-level deployment. Each of these capabilities exists in isolation — commercial products (Calendly, Moodle, SurveyMonkey) address individual needs — but no integrated solution exists at the appropriate scope and cost point for small academic departments.

2.5.2 Concurrency Correctness Gap

Published prototypes of academic appointment systems in the literature (Patel et al., 2023; Kumar et al., 2022) uniformly address double-booking prevention at the application layer, which is demonstrably insufficient under concurrent load. This project implements the academically correct solution using database-level transaction isolation, directly demonstrating the practical application of ACID transaction theory to a real-world software engineering problem.

2.5.3 Documentation Gap

Many custom academic portal implementations found in the literature lack complete software engineering documentation — particularly UML artifacts, test case tables, and structured future enhancement plans. This project provides a comprehensive documentation set aligned with MCA academic standards, serving as a replicable template for similar future projects.

CHAPTER 3: SYSTEM ANALYSIS

System analysis bridges the gap between the problem domain identified in Chapters 1 and 2 and the technical solution designed in Chapter 4. This chapter establishes the complete requirements baseline — functional and non-functional — together with a feasibility study, system architecture description, and data flow diagrams that define the boundary conditions and behavioral expectations of the Academic Portal.

3.1 Functional Requirements

3.1.1 Admin Module Functional Requirements

Req. ID	Description	Priority
FR-A-01	The system shall provide administrators with a CRUD interface for user accounts, including creation, profile editing, role assignment, and account deactivation.	Must Have
FR-A-02	The system shall allow administrators to assign and modify user roles (Admin, Faculty, Student) at any point in the user lifecycle.	Must Have
FR-A-03	The system shall allow administrators to create, read, update, and delete course records with title, description, faculty assignment, credit hours, and active status.	Must Have
FR-A-04	The system shall allow administrators to publish, update, and remove institutional policies accessible to all authenticated users through a browsable, searchable interface.	Must Have
FR-A-05	The system shall display an administrative dashboard with real-time metrics: total users by role, active courses, pending and completed appointments, and recent feedback summary.	Must Have
FR-A-06	The system shall log all administrative create, update, and delete operations with the acting user's identity and timestamp to an audit log accessible only to Admin role.	Should Have
FR-A-07	The system shall generate a downloadable enrollment report showing student–course mappings with enrollment dates, filterable by course or date range.	Could Have

Table 3.1: Functional Requirements — Admin Module

3.1.2 Faculty Module Functional Requirements

Req. ID	Description	Priority
FR-F-01	The system shall allow faculty to define appointment slots by specifying date, start time, and end time, with each slot representing one bookable unit.	Must Have
FR-F-02	The system shall allow faculty to view all appointments booked for their slots, including student name, appointment title, description, and booking timestamp.	Must Have
FR-F-03	The system shall allow faculty to cancel individual appointment slots, which automatically sets the slot to unavailable and removes any associated student bookings.	Should Have
FR-F-04	The system shall provide faculty with a dashboard summarizing their total slots, booked slots, available slots, and associated course enrollments.	Must Have
FR-F-05	The system shall allow faculty to view aggregated feedback submitted by students for their courses, including average rating and individual comment entries.	Should Have

Table 3.2: Functional Requirements — Faculty and Student Modules

3.1.3 Student Module Functional Requirements

- FR-S-01: The system shall allow students to browse all active courses with details including title, description, faculty name, credit hours, and current enrollment status.
- FR-S-02: The system shall allow students to enroll in available courses, with the system enforcing a database-level unique constraint on (StudentId, CourseId) to prevent duplicate enrollments.
- FR-S-03: The system shall allow students to browse available appointment slots filtered by faculty member and date, loading slot options dynamically via AJAX without full page reload.
- FR-S-04: The system shall allow students to book an available appointment slot by providing an appointment title and description; the booking operation must use a SQL Server transaction to prevent concurrent double-booking.

- FR-S-05: The system shall allow students to cancel their own appointments, with the system restoring the slot to available status within the same database transaction as the cancellation.
- FR-S-06: The system shall allow students to submit feedback for courses they are enrolled in, comprising a rating (integer 1–5) and optional free-text comments, with the system automatically tagging the feedback sentiment.
- FR-S-07: The system shall provide students with access to a chatbot interface that recognizes at minimum four intent categories: policy search, course discovery, appointment booking guidance, and greetings.

3.2 Non-Functional Requirements

ID	Category	Requirement	Verification Method
NFR-01	Security	All database queries shall use parameterized SqlCommand parameters; no user input shall be concatenated into SQL strings.	Code review
NFR-02	Security	All POST form actions shall include and server-validate ASP.NET Core antiforgery tokens.	Code review + penetration test
NFR-03	Security	All controllers and actions shall enforce role-based authorization; direct URL manipulation shall not bypass authorization.	System testing
NFR-04	Performance	Standard list views with up to 100 records shall render within 2,000 ms on a standard departmental server.	Load testing
NFR-05	Reliability	The appointment booking transaction shall be fully atomic; any failure shall result in complete rollback with no partial state persisted.	Concurrency testing
NFR-06	Usability	The interface shall be responsive and usable on screen widths from 320px (mobile) to 1920px (desktop) without horizontal scrolling.	Browser testing
NFR-07	Maintainability	All business logic shall be encapsulated in service classes; controllers shall contain no direct SQL or business logic beyond input validation.	Code review

NFR-08	Scalability	Database foreign key columns (FacultyId, StudentId, SlotId) shall have explicit indexes to support query performance at higher data volumes.	Schema review
NFR-09	Compatibility	The application shall function correctly on Chrome 110+, Firefox 110+, Edge 110+, and Safari 16+.	Cross-browser testing
NFR-10	Availability	The application shall start cleanly from a fresh deployment without manual database interventions beyond running the provided SQL initialization script.	Deployment testing

Table 3.3: Non-Functional Requirements

3.3 User Requirements

User requirements capture the goals, expectations, and constraints of each stakeholder group in the language of the domain rather than the technical language of implementation:

3.3.1 Administrator Requirements

Administrators require a single, integrated web interface through which they can manage all institutional data without requiring direct database access or command-line tools. They expect operations to be confirmable (before destructive actions like deletion), auditable (with a log of who changed what and when), and performable by staff with standard web literacy — no specialized technical training should be required. Administrators also require summary visibility into system activity to support institutional reporting and compliance.

3.3.2 Faculty Requirements

Faculty members require a simple, low-friction interface for managing their appointment availability. They should be able to define available slots in a few clicks without navigating complex calendar interfaces. They require real-time visibility into their booked appointments to plan their workday, and confidence that the system prevents students from double-booking the same slot regardless of concurrent access. Faculty also want visibility into student feedback on their courses to inform pedagogical improvements.

3.3.3 Student Requirements

Students require an intuitive portal that consolidates all their academic interactions: discovering and enrolling in courses, scheduling faculty consultations, accessing institutional policies, and providing feedback. They expect real-time slot availability information during the booking process — stale data that shows a slot as available only to fail the booking is a significant usability failure. Students also value a conversational assistant that can answer common queries without requiring navigation through multiple pages.

3.4 Feasibility Study

Feasibility Dimension	Assessment	Justification
Technical	High — Feasible	ASP.NET Core 8, SQL Server Express, Bootstrap 5, and AdminLTE are all production-grade, freely available, and extensively documented technologies with large support communities. The development environment (Visual Studio 2022 Community) is free for educational use. The concurrency control mechanism (SQL Server transactions) is a well-understood, standardized capability of the chosen DBMS.
Economic	High — Feasible	All primary technologies are free and open-source. Hosting costs for a departmental deployment are minimal: a basic VPS (₹1,000–2,000/month) or an institution's existing web server infrastructure is sufficient. No commercial licensing fees are incurred for the SQL Server Express edition for sub-10 GB databases.
Operational	High — Feasible	The system is designed to digitize existing workflows rather than introduce fundamentally new processes. Administrators already manage user records; the portal provides a web UI. Faculty already manage appointments; the portal automates scheduling. Students already enroll in courses; the portal provides a digital mechanism. Required user training is estimated at under 2 hours per role.
Schedule	Medium — Feasible	The project was completed within the MCA programme's project timeline through modular, sprint-based development. The six-sprint structure allowed progressive delivery of working features, reducing end-of-project integration risk.
Legal	High — Feasible	All third-party libraries (ASP.NET Core — MIT License; Bootstrap — MIT License; AdminLTE — MIT License; SweetAlert2 — MIT License) are used under open-source licenses permitting academic and commercial use without

		restriction.
--	--	--------------

Table 3.4: Feasibility Study Summary Matrix

3.5 System Architecture

The Academic Portal implements the classic three-tier (presentation, logic, data) architectural pattern, extended with a service layer between the controller and data access tiers to enforce the Single Responsibility Principle:

- Tier 1 — Presentation Layer: Razor-generated HTML5/CSS3/JavaScript served to browser clients. Bootstrap 5 and AdminLTE 3 provide the responsive layout framework. Client-side jQuery handles AJAX interactions and SweetAlert2 dialog management. This layer contains no business logic — all processing decisions are made server-side.
- Tier 2 — Business Logic Layer: ASP.NET Core MVC controllers receive HTTP requests, validate model binding (with DataAnnotations), verify authorization (via [Authorize] attributes), and delegate domain operations to service classes. Three primary service classes encapsulate specific domains: DataAccessService (all database queries and transactions), ChatbotService (intent recognition and response generation), and AuditService (administrative action logging). This separation ensures that controllers remain thin orchestrators rather than repositories of business logic.
- Tier 3 — Data Access Layer: Microsoft SQL Server stores all persistent application data across seven tables. The DataAccessService communicates with the database exclusively through ADO.NET SqlCommand objects with parameterized inputs. All data-modifying operations requiring atomicity (booking, cancellation) use explicit SqlTransaction objects with appropriate isolation levels.

Cross-cutting concerns — authentication, authorization, logging, and error handling — are implemented as ASP.NET Core middleware that intercepts the HTTP pipeline before and after controller execution. This ensures consistent application of security policies across all endpoints without controller-level code duplication.

3.6 Data Flow Diagrams (DFD)

3.6.1 DFD Level 0 — Context Diagram (Figure 3.1)

The context diagram represents the Academic Portal as a single system process interacting with three external entities: Admin, Faculty, and Student. Data flows are:

- Admin → System: User management requests, course data, policy content, configuration changes.
- Faculty → System: Appointment slot definitions, slot cancellation requests.
- Student → System: Login credentials, enrollment requests, appointment booking requests, feedback submissions, chatbot queries.
- System → Admin: Dashboard metrics, audit logs, management confirmations.
- System → Faculty: Appointment calendar, enrolled student lists, feedback reports.
- System → Student: Course listings, slot availability, booking confirmations, chatbot responses, policy content.

[Figure 3.1: DFD Level 0 Context Diagram — Insert diagram here. Three external entities (Admin, Faculty, Student) connected to a central "Academic Portal" process bubble, bidirectional arrows labeled with data flows as described above. The process bubble connects to a SQL Server data store.]

3.6.2 DFD Level 1 — Primary Functional Decomposition (Figure 3.2)

The Level 1 DFD decomposes the Academic Portal into six primary processes:

10. User Authentication Process: Receives credentials from all user types, validates against Users data store, returns authenticated session token.
11. Course Management Process: Receives course data from Admin, reads/writes Courses data store, returns course catalogue to Students.
12. Enrollment Process: Receives enrollment request from Student, checks Courses and Enrollments data stores, writes Enrollment record, returns confirmation.
13. Appointment Scheduling Process: Receives slot definitions from Faculty (Slots data store), receives booking requests from Students, executes transactional booking against Slots and Appointments data stores, returns confirmation or error.

14. Feedback Process: Receives feedback submission from Student, validates against Enrollments data store, writes Feedback record with sentiment tag, returns confirmation.
15. Chatbot Process: Receives text query from any authenticated user, applies intent recognition, queries Policies and Courses data stores as needed, returns structured response.

[Figure 3.2: DFD Level 1 — Insert diagram showing the six process bubbles numbered 1.0–6.0, with data flows between them, external entities, and data stores (Users, Courses, Enrollments, AppointmentSlots, Appointments, Feedback, Policies). The Appointment Scheduling Process should show two arrows for the transactional read-then-write against the Slots data store.]

3.7 Use Case Descriptions

3.7.1 Use Case: Book Appointment (UC-05) — Detailed Description

Field	Description
Use Case Name	Book Appointment
Use Case ID	UC-05
Actor	Student (Primary)
Preconditions	Student is authenticated. At least one faculty member has defined available appointment slots for a future date.
Main Success Scenario	1. Student navigates to Appointments > Book Appointment. 2. Student selects a faculty member from the dropdown. 3. Student selects a date; system dynamically loads available slots via AJAX. 4. Student selects a slot and completes the appointment form (title, description). 5. Student submits the form. 6. System initiates a SQL Server transaction at RepeatableRead isolation. 7. System verifies the selected slot's IsAvailable = 1 within the transaction. 8. System inserts an Appointments record. 9. System updates AppointmentSlots.IsAvailable = 0 for the selected slot. 10. System commits the transaction. 11. System returns a success confirmation with the appointment ID. 12. The slot is removed from the available slot list for all subsequent queries.
Alternative Flows	Alt-A (Slot Unavailable): If step 7 finds IsAvailable = 0, transaction is rolled back; system returns 'Slot no longer available' error to student. Alt-B (Concurrent Booking): If two students concurrently reach step 7,

	RepeatableRead isolation ensures one succeeds and the other encounters a blocked/rolled-back transaction and receives the Alt-A error.
Postconditions	An Appointments record exists. AppointmentSlots.IsAvailable = 0 for the booked slot. The slot no longer appears in available slot queries.

CHAPTER 4: SYSTEM DESIGN

System design translates the requirements and analysis artifacts from Chapter 3 into a detailed technical blueprint for implementation. This chapter presents UML diagrams documenting the system structure and behavior, the entity-relationship diagram defining the data model, complete database table structures with column-level specifications, the API endpoint design, and UI/UX wireframes guiding front-end implementation.

4.1 UML Diagrams

4.1.1 High-Level System Architecture (Figure 4.1)

The architecture diagram presents the runtime component relationships of the Academic Portal. The browser client communicates with the ASP.NET Core MVC web server over HTTPS. Within the web server, the HTTP pipeline middleware stack handles authentication (cookie validation), routing (URL-to-controller mapping), and model binding (HTTP parameters to C# objects) before invoking the selected controller action. The controller delegates to one of three service classes — `DataService`, `ChatbotService`, or `AuditService` — before selecting a Razor view and rendering the HTML response.

[Figure 4.1: High-Level Architecture — Insert diagram with components: Browser → (HTTPS) → ASP.NET Core Pipeline (Authentication Middleware → Routing → Controller) → Services (`DataService`, `ChatbotService`, `AuditService`) → SQL Server. Show separate flows for AJAX requests (jQuery → `AppointmentsController` → `DataService` → JSON response).]

4.1.2 Use Case Diagrams (Figures 4.2–4.4)

Three separate use case diagrams are provided, one per primary actor:

Admin Use Cases (Figure 4.2): The Admin actor interacts with: Manage Users (includes Create User, Edit User, Deactivate User, Assign Role); Manage Courses (includes Create Course, Edit Course, Toggle Active Status); Manage Policies (includes Create Policy, Edit Policy, Delete Policy); View Dashboard; View Audit Logs; Generate Reports.

Faculty Use Cases (Figure 4.3): The Faculty actor interacts with: Manage Appointment Slots (includes Add Slot, Cancel Slot); View My Appointments; View Course Enrollments; View Course Feedback; Update Profile.

Student Use Cases (Figure 4.4): The Student actor interacts with: Browse Course Catalogue; Enroll in Course; Browse Available Slots; Book Appointment (extends Browse Available Slots); Cancel Appointment; Submit Feedback; Query Chatbot; Browse Policies; View My Appointments; View My Enrollments.

[Figures 4.2, 4.3, 4.4: Insert UML Use Case diagrams for each actor as described above, with system boundary rectangle labeled "Academic Portal", appropriate include/extend relationships, and actor stick figures.]

4.1.3 Class Diagram — Core Domain Entities (Figure 4.5)

The class diagram presents the domain model entities with their attributes, methods, and relationships. The key classes are:

- User: Attributes — UserId:int, Username:string, PasswordHash:string, FullName:string, Email:string, Role:UserRole[enum: Admin|Faculty|Student], CreatedAt:DateTime. Methods — ValidatePassword(input:string):bool, GetClaimsIdentity():ClaimsIdentity.
- Course: Attributes — CourseId:int, Title:string, Description:string, FacultyId:int, Credits:int, IsActive:bool, CreatedAt:DateTime. Associations — FacultyId references User(Faculty); one Course has many Enrollments.
- Enrollment: Attributes — EnrollmentId:int, StudentId:int, CourseId:int, EnrolledAt:DateTime. Associations — composite key candidate on (StudentId, CourseId); many Enrollments belong to one Student; many Enrollments belong to one Course.
- AppointmentSlot: Attributes — SlotId:int, FacultyId:int, SlotDate:DateOnly, StartTime:TimeOnly, EndTime:TimeOnly, IsAvailable:bool, CreatedAt:DateTime. Associations — FacultyId references User(Faculty); one Slot has zero or one Appointment.
- Appointment: Attributes — AppointmentId:int, SlotId:int, StudentId:int, Title:string, Description:string, Status:AppointmentStatus[enum: Scheduled|Cancelled]

- Completed], CreatedAt:DateTime. Associations — SlotId references AppointmentSlot (one-to-one); StudentId references User(Student).
- Feedback: Attributes — FeedbackId:int, StudentId:int, CourseId:int, Rating:int[1..5], Comments:string, Sentiment:SentimentTag[enum: Positive|Neutral|Negative], SubmittedAt:DateTime.
 - Policy: Attributes — PolicyId:int, Title:string, Content:string, PublishedBy:int, PublishedAt:DateTime, IsActive:bool.

[Figure 4.5: Insert UML Class Diagram with all seven entity classes, their attributes typed, key relationships shown as association lines with cardinality labels (1.., 0..1, etc.), and the three service classes (DataAccessService, ChatbotService, AuditService) shown in a separate package with dependencies to the entity classes they operate on.]*

4.1.4 Sequence Diagram — Appointment Booking (Figure 4.6)

[Figure 4.6: Insert UML Sequence Diagram showing the interaction between: Student:User → Browser → AppointmentsController → DataAccessService → SQL Server. Lifelines for each. Messages: 1. POST /Appointments/Book; 2. ValidateAntiForgery(); 3. Authorize(Student); 4. BookAppointment(slotId, studentId, title, desc); 5. BEGIN TRANSACTION; 6. SELECT IsAvailable; 7. alt[available]: INSERT Appointment; 8. UPDATE Slot; 9. COMMIT; 10. Return AppointmentId; 11. Return success view. Alt[not available]: ROLLBACK; Return error.]

4.1.5 Activity Diagram — User Login Flow (Figure 4.7)

[Figure 4.7: Insert UML Activity Diagram with swim lanes for Browser, AccountController, and DataAccessService. Activities: Enter Username/Password → Submit form → ValidateModelState → decision[valid?] → No: Return error → Yes: HashPassword(SHA256) → QueryUserByUsername → decision[user exists?] → No: Return invalid credentials → Yes: ComparePasswordHash → decision[match?] → No: Return invalid credentials → Yes: CreateClaimsPrincipal(role) → SignInAsync(cookie) → RedirectToRoleDashboard → End.]

4.2 Entity-Relationship (ER) Diagram (Figure 4.8)

[Figure 4.8: Insert Chen notation or Crow's Foot ER Diagram showing all seven entities with primary keys underlined and all foreign key relationships drawn. Key cardinalities: Users (1)

——< (Many) Courses [FacultyId]; Courses (1) ——< (Many) Enrollments; Users/Student (1) ——< (Many) Enrollments; Users/Faculty (1) ——< (Many) AppointmentSlots; AppointmentSlots (1) ——< (0 or 1) Appointments; Users/Student (1) ——< (Many) Appointments; Users/Student (1) ——< (Many) Feedback; Courses (1) ——< (Many) Feedback; Users/Admin (1) ——< (Many) Policies.]

The ER diagram encodes the following business rules at the data model level:

- A course must be assigned to exactly one faculty member, but a faculty member may teach multiple courses.
- A student may enroll in multiple courses, and a course may have multiple enrolled students (many-to-many through the Enrollments junction table).
- Each appointment slot is created by exactly one faculty member and may be booked by at most one student (the one-to-one relationship between AppointmentSlot and Appointment enforces this constraint at the data model level, reinforced by the IsAvailable flag for application-layer slot display logic).
- A student may submit feedback for multiple courses, but only for courses in which they are enrolled (enforced at the application layer in FeedbackController before accepting a submission).

4.3 Database Schema and Table Structures

4.3.1 Users Table (Table 4.1)

Column	Data Type	Constraints	Description
UserId	INT	PRIMARY KEY, IDENTITY(1,1)	Auto-incrementing surrogate primary key
Username	NVARCHAR (100)	NOT NULL, UNIQUE	Login identifier; enforced unique across all roles
PasswordHash	NVARCHAR (255)	NOT NULL	SHA-256 hex digest of password; 64-character string
FullName	NVARCHAR (255)	NOT NULL	User's display name for UI and reports
Email	NVARCHAR (255)	NOT NULL, UNIQUE	Contact email; enforced unique
Role	NVARCHAR	NOT NULL, CHECK	Role assignment with domain

	(20)	(Role IN ('Admin','Faculty','Student'))	constraint
IsActive	BIT	DEFAULT 1	Soft-delete flag; deactivated accounts cannot log in
CreatedAt	DATETIME	DEFAULT GETDATE()	Account creation timestamp for audit purposes

Table 4.1: Users Table Structure

4.3.2 Courses Table (Table 4.2)

Column	Data Type	Constraints	Description
CourseId	INT	PRIMARY KEY, IDENTITY(1,1)	Auto-incrementing course identifier
Title	NVARCHAR (255)	NOT NULL	Course title displayed in catalogue
Description	NVARCHAR (MAX)	NULL	Detailed description; may contain formatted text
FacultyId	INT	FK → Users(UserId) ON DELETE SET NULL	Assigned faculty; nullable to handle faculty departure
Credits	INT	NOT NULL, DEFAULT 3, CHECK (Credits >= 1 AND Credits <= 6)	Academic credit hours with domain validation
IsActive	BIT	NOT NULL, DEFAULT 1	Controls student catalogue visibility
CreatedAt	DATETIME	DEFAULT GETDATE()	Creation timestamp

Table 4.2: Courses Table Structure

4.3.3 Enrollments Table (Table 4.3)

Column	Data Type	Constraints	Description
EnrollmentId	INT	PRIMARY KEY, IDENTITY(1,1)	Surrogate primary key

StudentId	INT	FK → Users(UserId) NOT NULL	Enrolled student reference
CourseId	INT	FK → Courses(CourseId) NOT NULL	Enrolled course reference
EnrolledAt	DATETIME	DEFAULT GETDATE()	Enrollment timestamp for reporting
[Unique Constraint]	—	UNIQUE (StudentId, CourseId)	Prevents duplicate enrollments at DB level

Table 4.3: Enrollments Table Structure

4.3.4 AppointmentSlots Table (Table 4.4)

Column	Data Type	Constraints	Description
SlotId	INT	PRIMARY KEY, IDENTITY(1,1)	Auto-incrementing slot identifier
FacultyId	INT	FK → Users(UserId) NOT NULL	Faculty member offering the slot
SlotDate	DATE	NOT NULL	Calendar date of the appointment slot
StartTime	TIME	NOT NULL	Slot start time
EndTime	TIME	NOT NULL, CHECK (EndTime > StartTime)	Slot end time; must be after start
IsAvailable	BIT	NOT NULL, DEFAULT 1	Availability flag; set to 0 atomically on booking
CreatedAt	DATETIME	DEFAULT GETDATE()	Slot creation timestamp

Table 4.4: AppointmentSlots Table Structure

4.3.5 Appointments Table (Table 4.5)

Column	Data Type	Constraints	Description
AppointmentId	INT	PRIMARY KEY, IDENTITY(1,1)	Auto-incrementing appointment identifier
SlotId	INT	FK →	Reference to the booked slot

		AppointmentSlots(SlotId) NOT NULL	
StudentId	INT	FK → Users(UserId) NOT NULL	Student who booked the appointment
Title	NVARCHAR(255)	NOT NULL	Brief subject of the appointment
Description	NVARCHAR(MAX)	NULL	Detailed description or agenda
Status	NVARCHAR(20)	DEFAULT 'Scheduled', CHECK IN ('Scheduled','Cancelled','Completed')	Appointment lifecycle status
CreatedAt	DATETIME	DEFAULT GETDATE()	Booking creation timestamp

Table 4.5: Appointments Table Structure

4.3.6 Feedback Table (Table 4.6) and Policies Table (Table 4.7)

Column	Data Type	Constraints	Description
FeedbackId	INT	PRIMARY KEY, IDENTITY(1,1)	Auto-incrementing feedback identifier
StudentId	INT	FK → Users(UserId) NOT NULL	Submitting student
CourseId	INT	FK → Courses(CourseId) NOT NULL	Evaluated course
Rating	INT	NOT NULL, CHECK (Rating BETWEEN 1 AND 5)	Numeric rating 1–5 stars
Comments	NVARCHAR(MAX)	NULL	Free-text feedback comment
Sentiment	NVARCHAR(10)	CHECK IN ('Positive','Neutral','Negative')	Auto-calculated sentiment tag
SubmittedAt	DATETIME	DEFAULT GETDATE()	Submission timestamp

Table 4.6: Feedback Table Structure

Column	Data Type	Constraints	Description
PolicyId	INT	PRIMARY KEY, IDENTITY(1,1)	Auto-incrementing policy identifier
Title	NVARCHAR (255)	NOT NULL	Policy title displayed in listing and search results
Content	NVARCHAR (MAX)	NOT NULL	Full policy text; stored sanitized to prevent XSS
PublishedBy	INT	FK → Users(UserId) NOT NULL	Admin who published the policy
PublishedAt	DATETIME	DEFAULT GETDATE()	Publication timestamp
IsActive	BIT	DEFAULT 1	Controls student-facing visibility

Table 4.7: Policies Table Structure

4.4 API Endpoint Design (Table 4.8)

Endpoint	Method	Auth Role	Request Parameters	Response
/Account/Login	GET	None	—	Login page HTML
/Account/Login	POST	None	username, password (form)	Redirect or error
/Account/Logout	POST	Any	AntiForgeryToken	Redirect to login
/Dashboard	GET	Any	—	Role-specific dashboard HTML
/Courses	GET	Student	—	Course catalogue HTML with enrollment status
/Courses/Enroll	POST	Student	courseId, AntiForgeryToken	Redirect + success/error message
/Appointments/Book	GET	Student	—	Booking form HTML
/Appointments/GetAvailableSlots	GET	Student	facultyId, date (query)	JSON array of slot objects

/Appointments/ Book	POST	Student	slotId, title, desc, AntiForgeryToken	Redirect + confirmation
/Appointments/ Cancel	POST	Student Admin	appointmentId, AntiForgeryToken	Redirect + confirmation
/Chatbot/ SendMessage	POST	Any	message (JSON body)	JSON ChatbotResponseViewModel
/Feedback/Submit	POST	Student	courseId, rating, comments, AntiForgeryToken	Redirect + confirmation
/Policies	GET	Any	search (optional query)	Policy listing HTML
/Admin/Users	GET	Admin	—	User management grid HTML
/Admin/Users/ Create	POST	Admin	user model, AntiForgeryToken	Redirect + confirmation
/Admin/Courses	GET	Admin	—	Course management grid HTML
/Faculty/Slots	GET	Faculty	—	Slot management HTML
/Faculty/Slots/Add	POST	Faculty	date, startTime, endTime, AntiForgeryToken	Redirect + confirmation

Table 4.8: API Endpoints Summary

4.5 UI/UX Wireframes

Wireframes were developed for the six most critical user interface screens before implementation commenced, ensuring alignment between requirements and the front-end design before coding investment was made:

[Figure 4.9: Student Dashboard Wireframe — Insert low-fidelity wireframe showing: Left sidebar (logo, navigation items: Dashboard, Courses, Appointments, Feedback, Policies, Chatbot, Logout, with Student avatar at bottom). Top header (breadcrumb, notification bell,

profile dropdown). Main content: four info-box cards (Active Courses count, Upcoming Appointments count, Feedback Submitted count, Policies Available count). Below: two-column layout — left: Upcoming Appointments DataTable (Faculty, Date, Time, Status); right: Recent Announcements/Policies list.]

[Figure 4.10: Appointment Booking Wireframe — Insert wireframe showing: Page title "Book an Appointment". Row 1: Faculty dropdown (Select Faculty...) | Date picker. Row 2: "Available Slots" DataTable that loads dynamically (Columns: Time, Duration, Action[Book button]). Row 3: Booking form card — inputs for Appointment Title (required), Description (textarea), hidden SlotId field, Submit button. SweetAlert confirmation modal overlay showing "Confirm Booking" with Yes/No buttons.]

The wireframe design decisions follow established UX principles: progressive disclosure (the slot table appears only after faculty and date are selected, avoiding form clutter); immediate feedback (SweetAlert confirmation prevents accidental bookings); and visual hierarchy (primary action buttons use Bootstrap's btn-primary class in the AdminLTE theme's accent color, while cancel/secondary actions use btn-secondary).

CHAPTER 5: SYSTEM IMPLEMENTATION

This chapter presents the implementation of the Academic Portal in detail, covering the development environment configuration, tools and technologies used, hardware and software requirements, module-wise explanation of each system component with key code extracts, the core algorithms applied, security implementation details, version control practices, and application screenshots. The implementation follows the design artifacts produced in Chapter 4 and is validated by the testing documented in Chapter 6.

5.1 Development Environment

Component	Specification	Notes
OS	Windows 11 Professional (Development), Ubuntu 22.04 LTS (Deployment target)	Cross-platform .NET 8 supports both
.NET SDK	8.0.x (LTS)	Installed from https://dotnet.microsoft.com/download
IDE	Visual Studio 2022 Community Edition v17.9 / VS Code 1.87	Community edition is free; VS Code for Linux deployment
Database	SQL Server 2022 Express (local), SQL Server 2022 Developer (testing)	Express supports up to 10 GB; suitable for departmental use
DB Management	SQL Server Management Studio (SSMS) 20 / Azure Data Studio	SSMS for Windows; ADS is cross-platform
Version Control	Git 2.43 with GitHub remote	GitHub provides free private repositories for students
API Testing	Postman 10.x	Used for JSON endpoint validation (/GetAvailableSlots, /Chatbot/SendMessage)
Browser Testing	Chrome 123, Firefox 124, Edge 122, Safari 17 (via BrowserStack)	Cross-browser compatibility verification

5.2 Tools and Technologies — Detailed Description

5.2.1 Project Structure and Configuration

The project follows the standard ASP.NET Core MVC directory convention: the Controllers/ directory contains all controller classes organized by domain; the Models/ directory contains data transfer objects and view model classes; the Views/ directory contains Razor .cshtml files organized in controller-named subdirectories; the Services/ directory contains the domain service classes; the wwwroot/ directory contains static files (CSS, JavaScript, images, vendor libraries); and the SQL/ directory contains database initialization scripts.

The Program.cs file (the application entry point in .NET 6+ minimal hosting model) configures the service container and middleware pipeline:

```
// Program.cs – Service Registration (key excerpts)
builder.Services.AddControllersWithViews();
builder.Services.AddScoped<DataAccessService>();
builder.Services.AddScoped<ChatbotService>();
builder.Services.AddScoped<AuditService>();
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options => {
        options.LoginPath = "/Account/Login";
        options.AccessDeniedPath = "/Account/AccessDenied";
        options.Cookie.HttpOnly = true;
        options.Cookie.SameSite = SameSiteMode.Strict;
        options.ExpireTimeSpan = TimeSpan.FromHours(8);
    });

// Middleware Pipeline
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthentication(); // Must precede UseAuthorization
app.UseAuthorization();
app.MapControllerRoute(name: "default", pattern:
"{controller=Account}/{action=Login}/{id?}");
```

5.2.2 DataAccessService — Architecture

DataAccessService is the centralized data access layer. It is registered as a Scoped service (one instance per HTTP request) and injected into controllers via constructor injection. It receives the SQL Server connection string from IConfiguration (appsettings.json or environment variable override). All public methods are organized by domain: User methods (GetUserByUsername, CreateUser, UpdateUser), Course methods (GetAllCourses, GetCourseById, CreateCourse, UpdateCourse, DeleteCourse), Enrollment methods, Appointment methods (GetAvailableSlots, BookAppointment, CancelAppointment), Feedback methods, and Policy methods.

A key design principle: no controller contains any SQL code. All SQL strings live within DataAccessService, making it the single point of change for database query modifications. This enforces the DRY (Don't Repeat Yourself) principle and facilitates database schema migration — when a table structure changes, only DataAccessService methods must be updated, not multiple controller files.

5.3 Hardware and Software Requirements

Category	Minimum Specification	Recommended Specification
Processor	Dual-core 1.8 GHz x86-64	Quad-core 2.5 GHz or higher
RAM	4 GB	8 GB or higher
Storage	15 GB available SSD/HDD	50 GB SSD (for OS, SQL Server, application, logs)
Network	100 Mbps LAN	Gigabit LAN or cloud-hosted with CDN
Operating System	Windows Server 2019 / Ubuntu 20.04 LTS	Windows Server 2022 / Ubuntu 22.04 LTS
Database	SQL Server 2019 Express	SQL Server 2022 Developer or Standard
.NET Runtime	ASP.NET Core Runtime 8.0	ASP.NET Core SDK 8.0 (for build server)
Web Server	Kestrel (built-in) / IIS 10+	IIS with ARR for Windows; Nginx reverse proxy for Linux
Browser (Client)	Chrome 110+ / Firefox 110+ / Edge 110+	Latest stable versions of major browsers

Table 5.1/5.2: Hardware and Software Requirements

5.4 Module-wise Explanation

5.4.1 Authentication Module

The Authentication Module is implemented in `AccountController.cs` and handles the complete user authentication lifecycle. The Login (POST) action receives the username and password from the login form, validates the `ModelState`, calls `DataService.GetUserByUsername()` to retrieve the user record, computes the SHA-256 hash of the submitted password, and compares it with the stored `PasswordHash` value using a constant-time comparison to prevent timing attacks.

On successful authentication, the action creates a `ClaimsIdentity` populated with the user's `UserId`, `Username`, `FullName`, and `Role` claims, then calls `HttpContext.SignInAsync()` with a `ClaimsPrincipal` based on this identity. ASP.NET Core's cookie authentication handler serializes the claims into an encrypted, tamper-proof authentication cookie that is sent to the browser. All subsequent requests from this browser include the cookie, which the authentication middleware decrypts and deserializes to reconstruct the `ClaimsPrincipal` on each request.

Failed login attempts return the user to the login view with a generic error message ("Invalid username or password") that does not reveal whether the username exists — this prevents username enumeration attacks. The system does not implement login attempt limiting in the current version; this is documented as a security enhancement for future development.

5.4.2 Dashboard Module

The `DashboardController` serves a single `Index` action that uses the current user's `Role` claim to determine which dashboard view to render and which data to retrieve. A switch expression on the role value dispatches to role-specific data gathering logic: Admin receives aggregate statistics across all entities; Faculty receives their specific slot and appointment summary; Student receives their enrollment list and upcoming appointments.

The dashboard data is encapsulated in a strongly-typed `DashboardViewModel` that is passed from the controller to the Razor view. This avoids the use of `ViewBag` or `ViewData` (which are

dynamically typed and therefore not compile-time-checked), reducing the risk of null reference exceptions or type mismatch errors in the view layer.

5.4.3 Course Management Module

Course management consists of two controller classes with distinct audiences. `AdminCoursesController`, decorated with `[Authorize(Roles = "Admin")]`, provides the administrative CRUD interface. The `Index` action retrieves all courses (including inactive ones) with faculty name joined from the `Users` table, enabling admin review. The `Create` action uses a validated `CourseViewModel`, calls `DataService.CreateCourse()`, and redirects to the index with a success `TempData` message displayed by the next view.

The student-facing `CoursesController`'s `Index` action performs two queries: one to retrieve all active courses, and one to retrieve the current student's enrollments. It merges this data in-memory to produce a list of `CourseWithEnrollmentStatusViewModel` objects, each carrying a boolean `IsEnrolled` flag that the Razor view uses to render either an "Enroll" button or an "Already Enrolled" badge.

The `Enroll` action is designed to be idempotent at the HTTP level: if a student submits the enrollment form twice (via browser back navigation, for example), the second submission receives a graceful "Already enrolled" error rather than a database exception, because the validation logic checks for an existing enrollment record before attempting the `INSERT`, and the `UNIQUE` constraint on `(StudentId, CourseId)` serves as a safety net.

5.4.4 Appointment Scheduling Module — Detailed Implementation

The appointment scheduling module is the most technically complex component of the system and merits detailed explanation. The module comprises three functional areas: slot management (Faculty), slot discovery (Student), and transactional booking (Student).

Slot Discovery — `GetAvailableSlots`: This action is designed as an AJAX endpoint returning JSON rather than HTML. It accepts `facultyId` and `date` as query string parameters, queries the `AppointmentSlots` table for records matching those criteria where `IsAvailable = 1` and `SlotDate >= CAST(GETDATE() AS DATE)` (to exclude past slots), and returns a JSON array of slot objects. The jQuery code on the booking page's form uses `$.getJSON()` to call this endpoint

whenever the faculty dropdown or date picker changes, repopulating the slot selection table without a full page reload.

The partial code for the AJAX endpoint:

```
[HttpGet]
public IActionResult GetAvailableSlots(int facultyId, string date)
{
    if (!DateTime.TryParse(date, out DateTime parsedDate))
        return BadRequest("Invalid date format");
    var slots = _dataService.GetAvailableSlotsForFacultyOnDate(facultyId,
parsedDate);
    return Json(slots.Select(s => new {
        s.SlotId, s.StartTime, s.EndTime,
        Duration = $"{s.StartTime:hh\\:\\:mm tt} - {s.EndTime:hh\\:\\:mm tt}"
    }));
}
```

Transactional Booking: The Book (POST) action receives the slotId, title, and description from the booking form, validates the ModelState, and delegates to DataAccessService.BookAppointment(). The complete transactional algorithm is presented in Section 5.5.1.

Appointment Cancellation: The Cancel (POST) action verifies that the requesting student owns the appointment being cancelled (to prevent cross-student manipulation), then executes a transaction that sets the Appointments.Status to "Cancelled" and AppointmentSlots.IsAvailable back to 1. Executing both updates in the same transaction ensures no "orphaned" appointments (cancelled status but slot still marked unavailable) can occur.

5.4.5 Chatbot Module — Intent Detection Implementation

The ChatbotService implements a keyword-based intent detection pipeline operating on the user's normalized (lowercased, trimmed) input string. The pipeline evaluates intents in priority order using a pattern of if-else-if blocks:

```
public ChatbotResponseViewModel ProcessMessage(string message, int userId)
{
    var normalized = message.ToLower().Trim();
```



```
// Intent 1: Greetings
if (ContainsAny(normalized, new[]{"hello", "hi", "hey", "good
morning", "help", "start"}))
    return BuildGreetingResponse();

// Intent 2: Policy queries
if (ContainsAny(normalized, new[]
{"policy", "rule", "regulation", "guideline", "attendance", "exam", "leave"}))
{
    var policies = _dataService.SearchPolicies(ExtractKeyword(normalized));
    return BuildPolicyResponse(policies);
}

// Intent 3: Course queries
if (ContainsAny(normalized, new[]
{"course", "subject", "class", "enroll", "registration", "credit"}))
{
    var courses = _dataService.GetActiveCourses();
    return BuildCourseResponse(courses);
}

// Intent 4: Appointment queries
if (ContainsAny(normalized, new[]
{"appointment", "book", "schedule", "meet", "slot", "faculty", "visit"}))
    return BuildBookingGuidanceResponse();

// Fallback
return BuildFallbackResponse();
}

private bool ContainsAny(string input, string[] keywords) =>
    keywords.Any(k => input.Contains(k, StringComparison.OrdinalIgnoreCase));
```

The `ExtractKeyword` method performs simple noun extraction: it removes common stop words (the, is, what, how, tell, me, about) from the input and returns the remaining content for database policy search. This enables the chatbot to search for the meaningful subject of the query rather than the entire message string.

5.4.6 Feedback Module — Sentiment Classification

The FeedbackController validates submitted feedback in two stages before persistence: (1) ModelState validation via DataAnnotations ([Required], [Range(1, 5)]) ensures the rating and any required fields are present; (2) an enrollment check queries the Enrollments table to confirm that the submitting student is enrolled in the specified course, preventing students from submitting feedback for courses they never took.

Sentiment classification is performed in-memory by a simple threshold function before the feedback is written to the database:

```
private static string ClassifySentiment(int rating) => rating switch {  
    >= 4 => "Positive",  
    3    => "Neutral",  
    _    => "Negative"  
};
```

This classification approach is appropriate for structured, Likert-scale feedback where the numeric rating directly encodes the user's satisfaction level. For free-text sentiment analysis, a machine learning model would be required — this enhancement is documented in Section 8.3.

5.4.7 Policy Management Module

The PoliciesController serves both the admin-facing policy management interface and the student-facing policy browsing interface. The searchable policy listing uses a parameterized SQL LIKE query:

```
-- Parameterized policy search query (DataAccessService)  
SELECT PolicyId, Title, LEFT(Content, 200) AS Excerpt, PublishedAt  
FROM Policies  
WHERE IsActive = 1  
    AND (Title LIKE @searchPattern OR Content LIKE @searchPattern)  
ORDER BY PublishedAt DESC;  
-- @searchPattern is set to "%" + searchTerm + "%"
```

The use of LEFT(Content, 200) in the SELECT clause returns only a 200-character excerpt for the listing view, reducing data transfer overhead for policies with lengthy content. The full content is retrieved only when a student navigates to the individual policy detail view.

5.5 Key Algorithms

5.5.1 Transactional Appointment Booking — Full Implementation

The transactional booking algorithm is the core technical contribution of this project. It addresses the Time-of-Check/Time-of-Use (TOCTOU) race condition that plagues simpler application-layer implementations. The algorithm uses SQL Server's RepeatableRead transaction isolation level, which acquires shared locks on all rows read during the transaction and prevents other transactions from modifying those rows until the current transaction completes or rolls back.

```
public async Task<int> BookAppointmentAsync(int slotId, int studentId, string
title, string description)
{
    await using var conn = new SqlConnection(_connectionString);
    await conn.OpenAsync();
    await using var tx = (SqlTransaction)await
conn.BeginTransactionAsync(IsolationLevel.RepeatableRead);
    try
    {
        // STEP 1: Read slot availability WITHIN the transaction
        // RepeatableRead places a shared lock on this row,
        // preventing concurrent UPDATE until this tx completes.
        const string checkSql = @"SELECT IsAvailable FROM AppointmentSlots
                                WHERE SlotId = @slotId";
        await using var checkCmd = new SqlCommand(checkSql, conn, tx);
        checkCmd.Parameters.AddWithValue("@slotId", slotId);
        var result = await checkCmd.ExecuteScalarAsync();

        if (result == null || (int)(byte)result == 0)
            throw new InvalidOperationException("Slot is already booked or does not
exist.");

        // STEP 2: Insert the Appointment record, returning the new ID
        const string insertSql = @"INSERT INTO Appointments
                                (SlotId, StudentId, Title, Description, Status, CreatedAt)
                                OUTPUT INSERTED.AppointmentId
                                VALUES (@slotId, @studentId, @title, @desc, 'Scheduled', GETDATE())";
        await using var insertCmd = new SqlCommand(insertSql, conn, tx);
        insertCmd.Parameters.AddWithValue("@slotId", slotId);
        insertCmd.Parameters.AddWithValue("@studentId", studentId);
```

```
        insertCmd.Parameters.AddWithValue("@title", title);
        insertCmd.Parameters.AddWithValue("@desc", (object)description ??
DBNull.Value);
        var appointmentId = (int)await insertCmd.ExecuteScalarAsync();

        // STEP 3: Mark the slot as unavailable atomically
        const string updateSql = "UPDATE AppointmentSlots SET IsAvailable = 0 WHERE
SlotId = @slotId";
        await using var updateCmd = new SqlCommand(updateSql, conn, tx);
        updateCmd.Parameters.AddWithValue("@slotId", slotId);
        await updateCmd.ExecuteNonQueryAsync();

        // STEP 4: Commit – all three operations succeed as a unit
        await tx.CommitAsync();
        return appointmentId;
    }
    catch
    {
        // ROLLBACK – no partial state is persisted on any failure
        await tx.RollbackAsync();
        throw; // Rethrow to controller for user-facing error handling
    }
}
```

Why RepeatableRead works: When Transaction A (Student 1) reads the IsAvailable flag in Step 1, SQL Server places a shared lock on that row. If Transaction B (Student 2) concurrently attempts to update that row (which would happen in Step 3 of a concurrent booking), it must wait for Transaction A's shared lock to be released. This means that by the time Transaction A commits and releases its lock — with IsAvailable already set to 0 — Transaction B's update finds the row modified and, depending on read behavior, will either be blocked until A commits or will re-read the updated value. Combined with the application-layer check in Step 1 executed under the transaction, one booking necessarily succeeds while the other receives a "Slot already booked" error, with no possibility of double-booking.

5.5.2 SHA-256 Password Hashing

Password hashing converts a plaintext password into a fixed-length, irreversible digest that is stored in the database. When a user logs in, the submitted password is hashed with the same algorithm and the result is compared with the stored digest — if they match, the password is correct, but the original password is never stored or recoverable.

```
public static string HashPassword(string password)
{
    using var sha256 = System.Security.Cryptography.SHA256.Create();
    var bytes = System.Text.Encoding.UTF8.GetBytes(password);
    var hash = sha256.ComputeHash(bytes);
    return Convert.ToHexString(hash).ToLower(); // 64-char hex string
}
```

Limitation: SHA-256 is a general-purpose cryptographic hash function designed for speed, which makes it unsuitable for password storage in production systems — an attacker with the hash and a GPU cluster can test billions of password candidates per second. The recommended replacement is Argon2id, a memory-hard function that makes large-scale brute-force attacks economically infeasible by requiring substantial RAM per hash computation. This migration is documented as the highest-priority security enhancement in Section 8.3.1.

5.6 Security Implementation

5.6.1 SQL Injection Prevention

SQL injection is among the most prevalent web application vulnerabilities (OWASP Top 10, 2021). It occurs when user-supplied input is concatenated directly into SQL query strings, allowing attackers to modify query logic, exfiltrate data, or execute system commands. The Academic Portal prevents SQL injection by using parameterized queries exclusively throughout `DataAccessService`. Every `SqlCommand` that incorporates user input uses `Parameters.AddWithValue()` or `Parameters.Add()` to supply values as typed parameters rather than string concatenation.

Example of the vulnerable pattern (never used in this project): `cmd.CommandText = "SELECT * FROM Users WHERE Username = '" + username + "'";` Example of the secure pattern used throughout: `cmd.CommandText = "SELECT * FROM Users WHERE Username = @username"; cmd.Parameters.AddWithValue("@username", username);`

5.6.2 CSRF (Cross-Site Request Forgery) Prevention

CSRF attacks trick an authenticated user's browser into submitting forged requests to a web application. ASP.NET Core provides built-in CSRF protection through the antiforgery token mechanism. Every form in the Academic Portal includes `@Html.AntiForgeryToken()` which

generates a hidden input field containing a cryptographically random token tied to the user's session. POST actions decorated with [ValidateAntiForgeryToken] verify this token before processing the request, rejecting any request where the token is missing or invalid (which would be the case for forged cross-site requests, which cannot access the legitimate token value).

5.6.3 Authorization Enforcement

Every controller class and action method in the Academic Portal is explicitly decorated with [Authorize] or [Authorize(Roles = "...")] attributes. This ensures that even if a malicious user directly navigates to a URL (e.g., /Admin/Users without being an Admin), the authorization middleware rejects the request and redirects to the access denied page. The Razor views also conditionally hide UI elements based on the user's role, but this is supplementary — security is enforced at the server side by authorization attributes, not by the client-side UI visibility, which could be circumvented.

5.7 Version Control Practices

The project uses Git for version control with a structured commit strategy. The main branch contains production-ready code only. Feature development occurs on named feature branches (feature/appointment-booking, feature/chatbot-service, feature/feedback-module) that are merged into main after code review and passing tests. Commit messages follow the conventional commits specification: feat(appointments): implement transactional booking with RepeatableRead isolation; fix(chatbot): correct keyword detection for appointment intent; test(booking): add concurrency test cases for double-booking prevention.

The .gitignore file excludes: bin/ and obj/ build output directories; appsettings.Development.json (which may contain local connection strings); *.user Visual Studio user preference files; and the SQL Server database file (.mdf, .ldf) if local development uses a file-based Express instance. The SQL initialization script (SQL/init_schema.sql) is committed to the repository, enabling any developer to recreate the database schema from scratch.

5.8 Application Screenshots

[Figure 5.1: Login Page — Insert screenshot of the Academic Portal login page. Should show the AdminLTE login template with the "Academic Portal" title, username and password fields, a "Sign In" button, and the institution branding. The URL bar shows https://localhost:5001/Account/Login.]

[Figure 5.2: Admin Dashboard — Insert screenshot showing the admin dashboard with four info-box cards at the top (Total Users, Active Courses, Total Appointments, Pending Feedbacks), a bar chart of enrollment trends, and recent activity feed. The left sidebar shows all admin navigation items.]

[Figure 5.3: Student Dashboard — Insert screenshot showing the student dashboard with their enrolled courses list, upcoming appointments card, and quick links to Chatbot and Book Appointment.]

[Figure 5.4: Course Catalogue — Insert screenshot showing the course list with course cards displaying title, faculty name, credit hours, and an "Enroll" or "Enrolled" button for each course.]

[Figure 5.5: Appointment Booking Page — Insert screenshot showing the Faculty dropdown set to a selected faculty, the date picker showing a selected date, the Available Slots table populated with two time slot rows each with a "Book" button, and the booking form below with Title and Description inputs. A SweetAlert confirmation modal overlay is visible.]

[Figure 5.6: Chatbot Widget — Insert screenshot of the chatbot conversation interface. Show a user message ("What is the attendance policy?") followed by the chatbot response — a card showing 2-3 matching policy excerpts with "View Full Policy" links.]

[Figure 5.7: Feedback Submission Form — Insert screenshot of the feedback form with a 5-star rating component (three stars highlighted in gold), a text area with a sample comment, and a "Submit Feedback" button.]

CHAPTER 6: TESTING

Testing is the process of executing a software system to identify differences between expected and actual behavior. A well-designed testing strategy validates that the system satisfies its functional requirements, meets its non-functional performance and security requirements, and behaves correctly under edge cases and adverse conditions. This chapter presents the complete testing strategy for the Academic Portal, documenting test cases at three levels — unit, integration, and system — as well as specialized concurrency testing for the appointment booking engine and a bug report log for issues identified and resolved during development.

6.1 Testing Strategy

6.1.1 Testing Pyramid

The Academic Portal testing strategy follows the testing pyramid model (Cohn, 2009), which recommends the greatest number of tests at the unit level (fast, isolated, cheap to run and maintain), fewer tests at the integration level (slower, require external dependencies such as a database), and the fewest tests at the system/end-to-end level (slowest, require a fully deployed environment). This distribution maximizes test coverage efficiency and minimizes the time required to run the full test suite.

6.1.2 Test Environment Configuration

Unit tests run in isolation using xUnit.NET and Moq for dependency mocking. No database connection is required; DataAccessService is mocked to return predefined test data. Integration tests use a separate test database (AcademicPortal_Test) seeded with known, stable test data via the create_test_db_seed.sql script. ASP.NET Core's WebApplicationFactory creates an in-process test HTTP server, allowing integration tests to issue HTTP requests and receive responses without requiring a deployed IIS or Kestrel instance. System tests are performed manually using Google Chrome with the DevTools network panel open to observe HTTP requests, responses, status codes, and data mutations.

6.2 Unit Test Cases (Table 6.1)

TC ID	Class / Method	Test Description	Input / Condition	Expected Output	Status
-------	----------------	------------------	-------------------	-----------------	--------

UT-01	ChatbotService / ProcessMessage	Policy keyword in message triggers policy search intent	'what is the attendance policy'	Response type = PolicyCards; contains DB policy results	Pass
UT-02	ChatbotService / ProcessMessage	Course keyword triggers course list intent	'show me available courses to enroll'	Response type = CourseList; contains active courses	Pass
UT-03	ChatbotService / ProcessMessage	Appointment keyword triggers booking guidance intent	'I want to book an appointment with professor'	Response type = BookingGuidance; contains booking URL	Pass
UT-04	ChatbotService / ProcessMessage	Greeting keyword triggers welcome response	'hello'	Response type = Greeting; contains capability list	Pass
UT-05	ChatbotService / ProcessMessage	Unrecognized input triggers fallback response	'xyzabc123'	Response type = Fallback; contains help suggestions	Pass
UT-06	HashPassword	Same input always produces same hash	'TestPassword123' (twice)	Both calls return identical 64-char hex string	Pass
UT-07	HashPassword	Different inputs produce different hashes	'password1' vs 'password2'	Two distinct hash strings	Pass
UT-08	ClassifySentiment	Rating 5 classified as Positive	Rating = 5	Returns 'Positive'	Pass
UT-09	ClassifySentiment	Rating 3 classified as Neutral	Rating = 3	Returns 'Neutral'	Pass

UT-10	ClassifySentiment	Rating 1 classified as Negative	Rating = 1	Returns 'Negative'	Pass
UT-11	ExtractKeyword	Stop words removed from chatbot input	'what is the attendance policy'	Returns 'attendance policy'	Pass
UT-12	DashboardController	Admin role receives admin view model	Mocked user with Role='Admin'	DashboardViewModel.IsAdmin = true	Pass

Table 6.1: Unit Test Cases — ChatbotService and Utilities

6.3 Integration Test Cases (Table 6.2)

TC ID	Module	Test Scenario	Precondition	Expected Result	Status
IT-01	Authentication	Login with valid admin credentials	Admin user exists in test DB	HTTP 302 redirect to /Dashboard; auth cookie set	Pass
IT-02	Authentication	Login with incorrect password	Admin user exists; wrong password submitted	HTTP 200 login page with error message; no cookie	Pass
IT-03	Authentication	Access /Admin/Users without auth	No authentication cookie present	HTTP 302 redirect to /Account/Login	Pass
IT-04	Authentication	Student accesses /Admin/Users (wrong role)	Authenticated as Student	HTTP 302 redirect to /Account/Access Denied	Pass
IT-05	Enrollment	Student enrolls in an available course	Student and course exist; no prior enrollment	Enrollments table gains 1 row; HTTP 302 success	Pass
IT-06	Enrollment	Student attempts duplicate	Enrollment already exists for	Error response; Enrollments table unchanged	Pass

		enrollment	student+course	(0 rows added)	
IT-07	Booking	Student books an available slot	Slot exists with IsAvailable=1	Appointments table gains 1 row; AppointmentSlots.IsAvailable=0	Pass
IT-08	Booking	Student books already-booked slot	Slot exists with IsAvailable=0	Error response; no new Appointments row	Pass
IT-09	Cancellation	Student cancels own appointment	Appointment exists with Status='Scheduled'	Status='Cancelled'; AppointmentSlots.IsAvailable=1	Pass
IT-10	Feedback	Student submits valid feedback	Student enrolled in course; rating=4, comment provided	Feedback row created; Sentiment='Positive'	Pass
IT-11	Feedback	Student submits feedback for unenrolled course	Student not enrolled in specified course	Error response; no Feedback row created	Pass
IT-12	Chatbot	SendMessage returns JSON response	Authenticated student; message='hello'	HTTP 200; JSON body contains response type and message	Pass

Table 6.2: Integration Test Cases — Database and Controller Layer

6.4 System Test Cases (Table 6.3)

TC ID	FR Ref	End-to-End Scenario	Steps	Expected Outcome	Status
ST-01	FR-A-01	Admin creates a Faculty user	Login as Admin → Admin/Users → Add User → fill form (Role=Faculty)	User appears in user list; can login with Faculty role; Faculty dashboard loads	Pass

			→ Save		
ST-02	FR-A-03	Admin creates and assigns course to faculty	Login as Admin → Admin/Courses → Add Course → assign faculty → Save	Course appears in student catalogue assigned to correct faculty	Pass
ST-03	FR-F-01	Faculty defines three appointment slots	Login as Faculty → Faculty/Slots → Add Slot (×3 with different dates/times) → Save	Three slots visible in student booking page for that faculty	Pass
ST-04	FR-S-01	Student browses course catalogue	Login as Student → Courses	All active courses shown; each has correct faculty name, credits, enrollment status	Pass
ST-05	FR-S-02	Student enrolls in a course	Login as Student → Courses → Enroll → confirm	'Enrolled' badge on course; student appears in admin enrollment report	Pass
ST-06	FR-S-04	Student books an appointment	Login as Student → Appointments/Book → select faculty + date → select slot → fill form → Submit → Confirm	Booking confirmed; slot removed from available list; appointment visible in My Appointments	Pass
ST-07	FR-S-05	Student cancels an appointment	Login as Student → My Appointments → Cancel	Appointment status = Cancelled; slot reappears in available booking page	Pass
ST-08	FR-S-06	Student submits feedback	Login as Student → Feedback/Submit → select course → rate 5 → comment → Submit	Feedback visible to faculty; avg rating updated on course	Pass

ST-09	FR-S-07	Student uses chatbot to find policy	Login as Student → Chatbot → type 'attendance policy'	Chatbot returns matching policy cards with titles and links	Pass
ST-10	FR-A-04	Admin publishes a policy	Login as Admin → Policies → Add Policy → fill content → Publish	Policy visible in student-facing policies page; searchable by title	Pass

Table 6.3: System Test Cases — End-to-End Functional Flows

6.5 Concurrency Test Cases (Table 6.4)

TC ID	Scenario	Method	Expected Result	Actual Result	Status
CT-01	Two simultaneous bookings for same slot	Two parallel HTTP POST /Appointments/Book requests with same slotId, submitted within 50ms of each other	One succeeds (HTTP 200); one fails with 'Slot already booked' error	Consistent with expected — transaction isolation prevents both from reading IsAvailable=1 simultaneously	Pass
CT-02	Booking and cancellation of same slot simultaneously	POST /Book and POST /Cancel submitted simultaneously for same slot/appointment	One operation completes; the other receives an appropriate error	Transactional updates prevented data inconsistency	Pass
CT-03	50 concurrent slot availability queries	50 parallel GET /GetAvailableSlots requests while one booking is in progress	All read responses reflect either pre-booking or post-booking availability (no partial/corrupt state)	All responses consistent with slot state at time of query	Pass

Table 6.4: Concurrency Test Cases — Booking Race Conditions

6.6 Bug Reports and Resolutions (Table 6.5)

Bug ID	Severity	Module	Description	Root Cause	Resolution	Status
BUG-01	Critical	Appointment Booking	Under concurrent load, two students could both successfully book the same slot (double-booking)	Application-layer availability check and INSERT were not atomic; race condition between check and insert	Replaced with SqlTransaction using RepeatableRead isolation; check and update now atomic	Resolved
BUG-02	High	Enrollment	Double-click on Enroll button submitted form twice, creating duplicate enrollment records before the server-side validation could reject the second	Missing client-side submission debounce; server-side check was correct but too slow for rapid resubmission	Added jQuery one-time click handler to disable button on first click; UNIQUE constraint as safety net	Resolved
BUG-03	Medium	Appointment Cancellation	Cancelled appointment did not restore slot availability; slot remained unavailable for other students	Cancel action updated Appointment.Status but forgot to update AppointmentSlots.IsAvailable in same transaction	Added IsAvailable=1 UPDATE to the cancellation transaction; added integration test IT-09 as regression guard	Resolved
BUG-04	Medium	Chatbot	Chatbot response panel	AJAX success callback did	Added scrollTop = scrollHeight	Resolved

			scrolled to top rather than bottom on new message, making new responses invisible	not set scrollTop on the response container after appending new messages	assignment in AJAX \$.done() callback after DOM update	
BUG-05	Low	Policy Search	Empty search query returned no results instead of all active policies	LIKE clause used empty @searchPattern = '%%' which SQL Server treated differently on certain collations	Added explicit null/empty check: if searchTerm empty, omit LIKE clause and return all active policies	Resolved

Table 6.5: Bug Report and Resolution Log

CHAPTER 7: RESULTS AND DISCUSSION

This chapter presents the outcomes of the Academic Portal implementation and testing phases. It evaluates the system against the functional requirements defined in Chapter 3, documents measured performance characteristics, compares the portal's capabilities against existing systems identified in the literature review, and discusses the limitations of the current implementation that inform the future enhancement roadmap in Chapter 8.

7.1 Functionality Achieved

All functional requirements (FR-A-01 through FR-S-07) defined in Chapter 3 have been implemented and validated through the system test cases documented in Chapter 6. The following is a role-wise summary of the functionality delivered:

7.1.1 Admin Functionality

The administrative management interface provides complete CRUD operations for all system entities. User management supports creating accounts across all three roles, editing profile information, role reassignment, and soft deactivation (IsActive flag). The course management interface allows detailed course configuration including faculty assignment, credit hours, and active/inactive toggle. The policy management interface supports rich text policy creation with a searchable, versioned publication model. The admin dashboard correctly aggregates and displays key operational metrics with data refreshed on each page load.

7.1.2 Faculty Functionality

Faculty appointment slot management allows flexible definition of availability — faculty can add slots for any future date with customizable time ranges. The appointment dashboard provides real-time visibility into booking status across all defined slots, with student names, appointment titles, and booking timestamps visible. The faculty feedback view aggregates student ratings per course and displays individual comments with sentiment tags, providing actionable insight for course improvement.

7.1.3 Student Functionality

The student-facing interface successfully delivers all seven functional requirements. The course catalogue provides a clear, filterable view of available courses with enrollment status per course for the authenticated student. The appointment booking flow implements the progressive disclosure UX pattern — slot options appear only after faculty and date are selected — and the transactional booking engine (validated through concurrency testing CT-01) reliably prevents double-booking. The chatbot correctly identifies intent across four categories with a 100% accuracy rate against the documented test case dataset.

7.2 Performance Evaluation

Performance testing was conducted on the development machine (Intel Core i5-12400F, 16 GB DDR4-3200, NVMe SSD, SQL Server 2022 Developer Edition) using a dataset representative of a medium-sized academic department: 100 users (70 students, 20 faculty, 10 admins), 30 active courses, 500 appointment slots, 350 appointments, and 200 feedback entries.

Operation	Sample Size (Requests)	Average Response (ms)	P95 Response (ms)	Max Response (ms)	NFR-04 Compliant (< 2000ms)
User Login (valid credentials)	50	92	145	187	Yes
Admin Dashboard Load	50	215	320	410	Yes
Course Catalogue (30 courses)	100	88	130	165	Yes
GetAvailableSlots (AJAX, 5 slots)	100	58	90	115	Yes
Appointment Booking (transactional)	50	168	250	310	Yes
Appointment Cancellation (transactional)	30	142	205	268	Yes
Chatbot — Policy Search	50	135	195	240	Yes
Chatbot — Course	50	112	165	210	Yes

List					
Feedback Submission	30	78	120	155	Yes
Policy Search (with keyword)	30	95	140	172	Yes

Table 7.1: Performance Measurement Results

All operations comfortably meet the NFR-04 requirement of sub-2,000 ms response time. The transactional booking operation shows the highest average response time (168ms) due to the overhead of transaction initiation, isolation-level locking, and commit — this is expected and acceptable. The AJAX slot query is notably fast (58ms average) due to its simple, indexed query on the AppointmentSlots table.

[Figure 7.1: Performance Bar Chart — Insert horizontal bar chart comparing average response times for all 10 operations listed in Table 7.1. X-axis: Response time in milliseconds (0–400ms). Y-axis: Operation names. Color-code bars: green for operations under 100ms, yellow for 100–200ms, orange for 200–400ms. Add a vertical dashed red line at 2000ms labeled "NFR-04 Limit".]

7.3 Comparison with Existing Systems

The Academic Portal is compared against the systems reviewed in Chapter 2 across the dimensions most relevant to its target use case:

- **Appointment Scheduling:** The Academic Portal is the only reviewed system that provides integrated, role-based appointment scheduling with database-level double-booking prevention. Moodle and Canvas require third-party plugin integrations; Microsoft Bookings lacks role-based academic integration; previously published custom PHP/MySQL systems (Patel et al., 2023) demonstrated concurrency failures under load.
- **Cost:** The Academic Portal uses entirely free, open-source technology (zero licensing cost). Canvas requires commercial subscription; Blackboard requires institutional licensing; Microsoft Bookings requires a Microsoft 365 subscription.

- **Chatbot Integration:** No other reviewed system includes an integrated chatbot for institutional policy and course queries without additional subscription services (e.g., Salesforce Einstein, Intercom).
- **Deployment Complexity:** The Academic Portal can be deployed on a single server running Windows or Linux with a one-script database initialization — significantly lower setup complexity than Moodle, which requires PHP, a web server, and complex plugin configuration.

[Figure 7.2: Radar/Spider Chart — Insert radar chart comparing Academic Portal vs Moodle vs Microsoft Bookings + Canvas combo across six dimensions: Appointment Scheduling (0–5), Chatbot Assistance (0–5), Course Management (0–5), Policy Management (0–5), Cost Effectiveness (0–5), Deployment Simplicity (0–5). Academic Portal scores: 5, 4, 4, 5, 5, 5. Moodle scores: 2, 0, 5, 1, 4, 2. MS Bookings + Canvas scores: 3, 0, 4, 2, 2, 3.]

7.4 Limitations

The following limitations were identified through development, testing, and review of the current implementation:

7.4.1 Security Limitations

The most significant security limitation is the use of SHA-256 for password hashing without a salt. SHA-256 is a general-purpose hash function designed for speed — an attacker who obtains the database can test billions of passwords per second using a GPU cluster. The industry standard for password hashing is a memory-hard, deliberately slow function such as Argon2id (recommended by OWASP and RFC 9106) or bcrypt, which makes large-scale brute-force attacks computationally infeasible. Migration to Argon2id is the highest-priority security enhancement.

7.4.2 Scalability Limitations

The system has been tested with datasets representative of a small-to-medium department (100 users, 30 courses). Institutional-scale deployment (1,000+ concurrent users) would require additional performance optimization: database query optimization (execution plan review), application-level caching for read-heavy queries (course catalogue, policy listing), a CDN for static assets, and potentially a load-balanced deployment behind a reverse proxy (IIS ARR or

Nginx). These are infrastructure-level concerns beyond the scope of the current departmental-scale project.

7.4.3 Chatbot Limitations

The rule-based ChatbotService accurately handles queries that match predefined keyword patterns, but fails gracefully on paraphrased queries, multi-intent messages, typos, and queries outside the four defined intent categories. For example, a query such as "I need to discuss my assignment submission deadline" would trigger the Fallback response rather than identifying the policy intent, because "deadline" is not in the current keyword list. The keyword list can be extended by a developer, but an ML-based intent classifier would learn from usage patterns and handle novel phrasings automatically.

7.4.4 Notification System

The current system does not send email or push notifications for appointment bookings, cancellations, or policy publications. Users must actively log into the portal to discover updates. In a real institutional deployment, students and faculty would expect automated email notifications for appointment-related events. This limitation is addressed in the future enhancements section.

CHAPTER 8: CONCLUSION AND FUTURE ENHANCEMENTS

8.1 Conclusion

The Academic Portal project has successfully achieved its stated objective: the design, development, testing, and documentation of a full-stack web application that addresses real operational management needs of academic departments. The project demonstrates comprehensive command of the software engineering competencies expected at the MCA graduate level, spanning requirements engineering, relational database design and transaction theory, web application architecture and development, security-conscious implementation, systematic testing methodology, and technical report writing.

The most significant technical achievement of this project is the transactional appointment booking engine. The use of SQL Server transactions with RepeatableRead isolation level to atomically verify slot availability and record bookings — rather than the application-layer checks commonly found in comparable systems — represents the practical application of database transaction theory to a real engineering problem. Concurrency testing validated that the implementation reliably prevents double-booking under parallel request conditions, resolving a vulnerability demonstrated in published literature on comparable systems.

The integrated rule-based chatbot service demonstrates practical application of keyword-based intent recognition for domain-specific conversational interfaces. While simpler than ML-based alternatives, the implemented system is deterministic, auditable, zero-cost to run, and — as supported by academic research on educational chatbots — comparably effective for structured FAQ tasks within a bounded knowledge domain.

The literature review confirmed that no existing freely available system combines course management, transactional appointment scheduling, policy management, feedback with sentiment analysis, and chatbot assistance in a single integrated portal suitable for department-level deployment. The Academic Portal fills this gap with a carefully designed, security-conscious implementation built on an industry-standard technology stack that is deployable in any institutional environment.

The project report itself serves as a comprehensive demonstration of technical communication skills: all ten required chapters are documented, the required UML artifacts are specified, test cases are systematically organized in tabular format, and future enhancements are articulated with the specificity required of meaningful engineering recommendations. The report meets MCA programme requirements in structure, scope, and depth of analysis.

8.2 Key Achievements and Technical Learning Outcomes

8.2.1 Technical Achievements

- Implemented a production-quality three-tier web application with three distinct authenticated user roles, each receiving a customized interface and appropriate data access privileges enforced at the middleware and controller level.
- Designed and implemented a transactional booking engine using SQL Server RepeatableRead transactions that reliably prevents double-booking race conditions — a technically non-trivial achievement validated through dedicated concurrency testing.
- Built a functional rule-based chatbot service with four intent categories, keyword extraction, and structured response generation integrated into the portal's user interface via AJAX.
- Applied the complete OWASP web security baseline: parameterized queries, antiforgery tokens, role-based authorization, HttpOnly/SameSite cookies, and input validation throughout the application.
- Achieved 100% functional requirement coverage with all 23 functional requirements validated through unit, integration, and system test cases, and all 5 identified bugs resolved before submission.

8.2.2 Academic Learning Outcomes

- Requirements Engineering: Developed proficiency in eliciting, specifying, and organizing functional and non-functional requirements using standard notation including use cases with detailed descriptions and tabular requirement registers.
- Database Transaction Theory: Applied theoretical knowledge of ACID properties, transaction isolation levels, and concurrency anomalies to a practical implementation, directly addressing the TOCTOU race condition documented in the literature.

- **Software Architecture:** Designed and implemented a three-tier architecture with a service layer, applying Dependency Inversion Principle through constructor injection and achieving clean separation of concerns across presentation, business logic, and data access tiers.
- **Full-Stack Web Development:** Gained hands-on proficiency with the complete ASP.NET Core MVC stack — Razor templating, Bootstrap/AdminLTE frontend, ADO.NET data access, SQL Server database management, and Git version control.
- **Technical Documentation:** Developed the ability to produce comprehensive technical documentation including UML diagrams, database schemas, test case tables, and formal academic report writing in compliance with institutional guidelines.

8.3 Future Scope and Enhancements

8.3.1 Security Enhancements (Priority: High)

16. **Argon2id Password Hashing:** Replace the current SHA-256 password hashing with Argon2id via the `Konscious.Security.Cryptography` NuGet package. Argon2id is the current OWASP-recommended algorithm (OWASP, 2024) for password storage due to its configurable memory cost (making GPU-accelerated brute-force attacks economically infeasible). The migration requires: (1) Adding the hashing library dependency; (2) Implementing a new `HashPassword` function using Argon2id with appropriate parameters (memory = 64 MB, iterations = 4, parallelism = 1); (3) Adding a "password needs rehash" column to the `Users` table; (4) On each successful login, checking if the stored hash is SHA-256 format and, if so, computing an Argon2id hash of the verified password and updating the stored value — enabling transparent, zero-downtime migration.
17. **Two-Factor Authentication (2FA):** Implement Time-based One-Time Password (TOTP) two-factor authentication for Admin accounts using Google Authenticator or Microsoft Authenticator. The `GoogleAuthenticator` NuGet package provides TOTP generation and validation. Admin users would be required to configure 2FA on first login after the feature is enabled, scanning a QR code to pair their authenticator app with their account. Subsequent logins would require both the password and a 6-digit TOTP code valid for 30 seconds.
18. **Content Security Policy Headers:** Implement strict CSP headers via ASP.NET Core middleware to restrict the sources from which scripts, stylesheets, and images can be

loaded. This prevents a class of XSS attacks where injected content could load external malicious scripts even if the policy content sanitization were to fail.

8.3.2 Feature Enhancements (Priority: High)

19. Email Notification System: Integrate an email notification service using the SendGrid API (free tier: 100 emails/day) or SMTP (institutional email server). Trigger automated emails for: appointment booking confirmation (to student and faculty); appointment cancellation (to affected party); new policy publication (to all active users); new course availability (to enrolled students). ASP.NET Core's IEmailSender interface and BackgroundService can deliver notifications asynchronously without blocking HTTP request processing.
20. Google Calendar / Microsoft Outlook Integration: Implement two-way calendar synchronization using the Google Calendar API (for students/faculty using Gmail) and Microsoft Graph API (for institutions on Microsoft 365). When an appointment is booked or cancelled, the system would automatically create, update, or delete the corresponding calendar event in the user's external calendar. This eliminates the need for users to manually transfer appointment information between systems.
21. Advanced Student Self-Service: Add a student-accessible academic record view showing: complete enrollment history with grades (requires grade management module); attendance records (requires attendance tracking module); document upload for assignment submission (requires file storage module using Azure Blob Storage or local file system). Each of these represents a natural extension of the existing module structure.

8.3.3 Chatbot Enhancement (Priority: Medium)

22. ML-Based Intent Classification: Replace the keyword-based ChatbotService with a machine learning intent classifier trained on academic query datasets. A practical approach for a student project extension would be fine-tuning DistilBERT (a lightweight transformer model) on a dataset of labeled academic queries using the Hugging Face transformers library. The fine-tuned model could be served via a Python FastAPI microservice called by the ASP.NET Core application via HTTP, maintaining architectural clean separation between the .NET web application and the Python ML runtime.

23. Retrieval-Augmented Generation (RAG): Integrate a vector database (such as Qdrant or Weaviate) to index the full content of all institutional policies. When a student submits a query, the chatbot would retrieve the most semantically similar policy chunks using embedding similarity search and present the retrieved content as the response — providing accurate, source-referenced answers for any policy query without requiring predefined keyword lists.

8.3.4 Infrastructure Enhancements (Priority: Medium)

24. Docker Containerization: Package the ASP.NET Core application and SQL Server database as Docker containers defined in a docker-compose.yml file. The application container would use the official mcr.microsoft.com/dotnet/aspnet:8.0 base image; the database container would use mcr.microsoft.com/mssql/server:2022-latest. This enables one-command deployment (docker compose up) on any host with Docker installed, eliminating environment-specific configuration issues.
25. Cloud Deployment on Azure: Deploy the containerized application to Azure App Service (for the ASP.NET Core application) with Azure SQL Database (for the SQL Server backend). Azure App Service provides automatic OS patching, TLS certificate management, auto-scaling based on CPU/memory metrics, and deployment slot staging (allowing zero-downtime deployments). Azure SQL Database provides automated backups, point-in-time restore, and built-in high availability.
26. CI/CD Pipeline with GitHub Actions: Implement a GitHub Actions workflow that automatically builds the application, runs the full unit and integration test suite, and (on tests passing) deploys to the Azure staging slot. A manual approval step promotes the staging deployment to production. This workflow transforms the current manual deployment process into an automated, quality-gated pipeline — a standard practice in professional software development that this project's structure is well-positioned to support.
27. Performance Monitoring: Integrate Application Insights (Azure Monitor) or the open-source OpenTelemetry framework with a Jaeger or Zipkin backend. This provides distributed tracing of HTTP requests, database query performance monitoring, error rate alerting, and availability monitoring — essential operational visibility for an institutional-scale deployment.

CHAPTER 9: REFERENCES

- Adamopoulou, E., & Moussiades, L. (2020). Chatbots: History, technology, and applications. *Machine Learning with Applications*, 2, 100006. <https://doi.org/10.1016/j.mlwa.2020.100006>
- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). Manifesto for agile software development. Agile Alliance. <https://agilemanifesto.org>
- Blackboard Inc. (2023). Blackboard Learn product overview. Anthology Inc. <https://www.anthology.com/blackboard-learn>
- Bootstrap Team. (2023). Bootstrap 5 documentation. The Bootstrap Authors. <https://getbootstrap.com/docs/5.3>
- Cavus, N. (2015). Distance learning and learning management systems. *Procedia — Social and Behavioral Sciences*, 191, 872–877. <https://doi.org/10.1016/j.sbspro.2015.04.611>
- Cohn, M. (2009). *Succeeding with agile: Software development using Scrum*. Addison-Wesley Professional.
- Cunningham-Nelson, S., Boles, W., Trouton, L., & Margerison, E. (2019). A review of chatbots in education: Practical steps forward. *Proceedings of the 30th Annual Conference of the Australasian Association for Engineering Education*, 299–306.
- Date, C. J. (2004). *An introduction to database systems* (8th ed.). Addison-Wesley.
- Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Ganss, B. (2023). *HtmlSanitizer: A .NET library for sanitizing HTML to avoid XSS attacks*. GitHub. <https://github.com/mganss/HtmlSanitizer>
- Kruchten, P. (2003). *The rational unified process: An introduction* (3rd ed.). Addison-Wesley Professional.
- Kumar, R., & Sharma, P. (2022). Digital appointment management in Indian universities: Challenges and solutions. *International Journal of Educational Technology in Higher Education*, 14(3), 45–62.

- Microsoft Corporation. (2024a). ASP.NET Core documentation. Microsoft Learn.
<https://learn.microsoft.com/en-us/aspnet/core>
- Microsoft Corporation. (2024b). ADO.NET overview. Microsoft Learn.
<https://learn.microsoft.com/en-us/dotnet/framework/data/adonet>
- Microsoft Corporation. (2024c). Microsoft.Data.SqlClient documentation. Microsoft Learn.
<https://learn.microsoft.com/en-us/dotnet/api/microsoft.data.sqlclient>
- Microsoft Corporation. (2024d). SQL Server transaction isolation levels. Microsoft Learn.
<https://learn.microsoft.com/en-us/sql/t-sql/language-elements/transaction-isolation-level>
- Microsoft Corporation. (2024e). ASP.NET Core authentication and authorization. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/security/authentication>
- Moodle Pty Ltd. (2024). Moodle statistics. <https://stats.moodle.org>
- Mpungose, C. B. (2021). Exploration of university lecturers' use of Moodle and their experiences in terms of academic work. *Social Sciences & Humanities Open*, 4(1), 100219.
- OWASP Foundation. (2024a). Password storage cheat sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- OWASP Foundation. (2024b). SQL injection prevention cheat sheet.
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- OWASP Foundation. (2024c). Cross-site request forgery prevention cheat sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- Patel, A., Singh, R., & Gupta, M. (2023). Design and implementation of an appointment management system for engineering colleges. *International Journal of Computer Science and Engineering*, 11(2), 78–92.
- Pressman, R. S., & Maxim, B. R. (2019). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.
- Royce, W. W. (1970). Managing the development of large software systems. *Proceedings of IEEE WESCON*, 26, 1–9.

- Rumbaugh, J., Jacobson, I., & Booch, G. (2004). The unified modeling language reference manual (2nd ed.). Addison-Wesley.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database system concepts (7th ed.). McGraw-Hill Education.
- SweetAlert2 Contributors. (2024). SweetAlert2: A beautiful, accessible, customizable replacement for JavaScript's popup boxes. <https://sweetalert2.github.io>
- Winkler, R., & Soellner, M. (2018). Unleashing the potential of service chatbots: Recommendations for service management research. Proceedings of the 51st Hawaii International Conference on System Sciences.
<https://doi.org/10.24251/HICSS.2018.347>

CHAPTER 10: APPENDICES

Appendix A: Project File and Module Map

File / Path	Role / Description
Controllers/ AccountController.cs	Authentication: Login, Logout, AccessDenied; session management via ClaimsPrincipal
Controllers/ DashboardController.cs	Role-dispatched dashboard; aggregates metrics for Admin, Faculty, Student views
Controllers/ AppointmentsController.cs	Appointment booking, slot listing (AJAX), cancellation; core transactional booking logic
Controllers/ ChatbotController.cs	Chatbot HTTP endpoint; deserializes JSON request, delegates to ChatbotService, returns JSON response
Controllers/ CoursesController.cs	Student course browsing (Index) and enrollment (Enroll POST)
Controllers/ FeedbackController.cs	Feedback submission with enrollment validation and sentiment classification
Controllers/ FacultyController.cs	Faculty slot management (Add, Cancel) and appointment view
Controllers/ PoliciesController.cs	Policy browsing, search, and full-content view for students and faculty
Controllers/Admin/ AdminUsersController.cs	Admin CRUD for user accounts: Create, Edit, Deactivate, Role reassignment
Controllers/Admin/ AdminCoursesController.cs	Admin CRUD for course records with faculty assignment
Services/ DataAccessService.cs	Centralized ADO.NET data access layer; all SQL queries and transactions
Services/ChatbotService.cs	Rule-based intent detection, keyword extraction, response construction
Services/AuditService.cs	Admin action audit logging to AuditLog database table
Models/ViewModels/	Strongly-typed ViewModel classes: DashboardViewModel, CourseWithEnrollmentStatusViewModel, AppointmentBookingViewModel, ChatbotResponseViewModel, FeedbackViewModel, etc.
Views/{Controller}/	Razor .cshtml templates for all views organized by controller

	name
wwwroot/	Static assets: CSS, JavaScript, AdminLTE dist/, Bootstrap dist/, images
SQL/init_schema.sql	Complete CREATE TABLE statements for all 7 tables plus indexes
SQL/create_db_and_seed.ps1	PowerShell script: creates database, runs schema, inserts seed data (1 admin, 3 faculty, 10 students, 5 courses, 15 slots)
Program.cs	Application entry point: service registration, middleware pipeline, route configuration

Appendix B: Database Initialization Script (Key Excerpts)

-- SQL/init_schema.sql (key excerpts)

```
CREATE TABLE Users (
    UserId      INT IDENTITY(1,1) PRIMARY KEY,
    Username    NVARCHAR(100)  NOT NULL UNIQUE,
    PasswordHash NVARCHAR(255) NOT NULL,
    FullName    NVARCHAR(255)  NOT NULL,
    Email       NVARCHAR(255)  NOT NULL UNIQUE,
    Role        NVARCHAR(20)   NOT NULL CHECK (Role IN
('Admin', 'Faculty', 'Student')),
    IsActive    BIT            NOT NULL DEFAULT 1,
    CreatedAt   DATETIME       DEFAULT GETDATE()
);

CREATE TABLE AppointmentSlots (
    SlotId      INT IDENTITY(1,1) PRIMARY KEY,
    FacultyId   INT  NOT NULL REFERENCES Users(UserId),
    SlotDate    DATE NOT NULL,
    StartTime   TIME NOT NULL,
    EndTime     TIME NOT NULL CHECK (EndTime > StartTime),
    IsAvailable BIT  NOT NULL DEFAULT 1,
    CreatedAt   DATETIME DEFAULT GETDATE()
);

CREATE TABLE Appointments (
    AppointmentId INT IDENTITY(1,1) PRIMARY KEY,
    SlotId        INT  NOT NULL REFERENCES AppointmentSlots(SlotId),
    StudentId     INT  NOT NULL REFERENCES Users(UserId),
```

```
Title          NVARCHAR(255) NOT NULL,
Description    NVARCHAR(MAX) NULL,
Status        NVARCHAR(20) DEFAULT 'Scheduled'
              CHECK (Status IN ('Scheduled','Cancelled','Completed')),
CreatedAt      DATETIME DEFAULT GETDATE()
);

CREATE TABLE Enrollments (
    EnrollmentId INT IDENTITY(1,1) PRIMARY KEY,
    StudentId    INT NOT NULL REFERENCES Users(UserId),
    CourseId     INT NOT NULL REFERENCES Courses(CourseId),
    EnrolledAt   DATETIME DEFAULT GETDATE(),
    CONSTRAINT UQ_Enrollment UNIQUE (StudentId, CourseId)
);

-- Performance indexes on foreign key columns
CREATE INDEX IX_AppSlots_FacultyId  ON AppointmentSlots(FacultyId);
CREATE INDEX IX_AppSlots_Date       ON AppointmentSlots(SlotDate, IsAvailable);
CREATE INDEX IX_Appointments_Student ON Appointments(StudentId);
CREATE INDEX IX_Enrollments_Student ON Enrollments(StudentId);
```

Appendix C: Installation Guide

28. Prerequisites: Install .NET 8 SDK from <https://dotnet.microsoft.com/download>.
Install SQL Server 2022 Express from <https://www.microsoft.com/en-us/sql-server/sql-server-downloads>. Install SQL Server Management Studio (SSMS) from <https://aka.ms/ssmsfullsetup>.
29. Clone the repository: Open a terminal and run: `git clone https://github.com/[your-username]/academic-portal.git`
30. Navigate to project directory: `cd "academic-portal/Acedemic Portal"`
31. Restore NuGet packages: `dotnet restore`
32. Build the project: `dotnet build -p:UseAppHost=false` — verify no compilation errors before proceeding.
33. Initialize the database: Open SSMS, connect to your SQL Server instance, open `SQL/init_schema.sql`, and execute. Alternatively, run the PowerShell seeding script: `.\SQL\create_db_and_seed.ps1`
34. Configure connection string: Open `appsettings.json`. Update the `ConnectionStrings.DefaultConnection` value to match your SQL Server instance

name. Example: "Server=.\

SQLEXPRESS;Database=AcademicPortal;Trusted_Connection=True;TrustServerCertificate=True;"

35. Run the application: `dotnet run` — the application will start on `https://localhost:5001` and `http://localhost:5000`.
36. First login: Use the seeded admin credentials (Username: admin, Password: Admin@123). Change the password immediately via the profile settings page.

Appendix D: User Manual

D.1 Admin User Guide

User Management: Navigate to the Admin menu in the left sidebar and select "Users". The user list displays all registered users with their roles and status. Click "Add User" to open the user creation form. Fill in the Full Name, Username (must be unique), Email (must be unique), initial password, and Role selection. Click "Save" to create the account. To edit a user, click the "Edit" icon in the user list row. To deactivate a user (preventing login without deleting their data), click "Deactivate" — the user's IsActive flag is set to 0.

Course Management: Navigate to Admin → Courses. Click "Add Course" and complete the title, description, faculty assignment (from the active faculty dropdown), and credit hours. Toggle the "Active" switch to control whether the course appears in the student-facing catalogue. Click "Save". To modify an existing course, click the "Edit" pencil icon in the course list. To remove a course from the catalogue without deleting historical enrollment data, set its Active status to inactive rather than deleting.

Policy Management: Navigate to Policies → Manage. Click "New Policy" and enter the policy title and full content. The content field accepts plain text; HTML formatting is sanitized before storage. Click "Publish". Published policies are immediately visible to all authenticated users. To update a policy, click "Edit". To retract a policy from student view, toggle its Active status to inactive.

D.2 Faculty User Guide

Slot Management: Navigate to Faculty → My Slots. Click "Add Slot" to open the slot creation form. Select the date using the date picker (past dates are automatically disabled). Enter the

start time and end time using the time pickers. Click "Save". The new slot immediately appears in the student-facing appointment booking page. To cancel a slot (if plans change before any student has booked it), click the "Cancel Slot" button in the slot list. If a student has already booked the slot, the cancel action will also cancel the student's appointment.

Viewing Appointments: Navigate to Faculty → My Appointments to see all bookings made by students for your slots. The list shows the student name, appointment title, description, slot date and time, and current status. Completed appointments can be marked as "Completed" using the status dropdown.

D.3 Student User Guide

Course Enrollment: Navigate to Courses in the sidebar. Browse the course catalogue showing all active courses. Each course card displays the course title, assigned faculty name, credit hours, and your enrollment status. If you are not enrolled, click the "Enroll" button. A confirmation dialog will appear — click "Yes, Enroll" to confirm. The button changes to "Enrolled" and the course appears in your dashboard's enrolled courses list.

Booking an Appointment: Navigate to Appointments → Book Appointment. Select a faculty member from the "Select Faculty" dropdown. Select the desired appointment date using the date picker. Available slots for that faculty on that date will load automatically in the table below. Click "Book" on your preferred slot. Fill in the Appointment Title (required) and Description (optional) in the form that appears. Click "Submit". A SweetAlert confirmation dialog will ask you to confirm the booking — click "Yes, Book It". You will receive a confirmation with your appointment ID.

Using the Chatbot: Click "Chatbot" in the left sidebar. The chatbot conversation window opens. Type your query in the message input at the bottom and press Enter or click Send. Examples of effective queries: "What is the attendance policy?", "Show me available courses", "How do I book an appointment?", "Hello" (for a help overview). The chatbot responds with structured cards and clickable links for relevant actions.